1 计算机系统概述

1.1 计算机系统层次结构

1. 电子管时代------晶体管时代（高级语言及其编译程序，磁芯存储器）------中小规模集成电路时代（半导体存储器）------超大规模集成电路时代（微处理器）
2. 1存储体------n个存储单元（与地址一一对应，n=2^MAR位数）------n×m个存储元件（存一个1/0，m为存储字长=MDR位数）
3. 汇编程序：汇编语言程序 → 机器语言程序

   解释程序：源程序 → 机器指令　顺序逐条翻译、立即执行

   编译程序：高级语言程序 → 汇编/机器语言程序　全部一次性翻译，产生可执行文件
4. 软件与硬件之间的界面：指令集体系结构 ISA，定义了一台计算机可以执行的所有指令的集合，称为软件可见部分
5. CPU包括：ALU + 通用寄存器组GPRs + 标志寄存器PSW + 控制器 + 程序计数器PC + MAR + MDR。
6. 源程序.c：预处理阶段（处理.h） → 源程序.i：编译阶段（编译） → 汇编源程序.s：汇编阶段（翻译成机器语言指令） → 可重定位目标文件.o：链接阶段（和库函数合并） → 可执行文件.exe 保存到磁盘
7. 冯诺依曼机的基本工作方式是控制流驱动方式，也就是按照指令的执行序列，依次读取指令，然后根据指令所含的控制信息，调用数据信息进行处理，因此，在执行程序的过程中，始终以控制信息流为驱动工作的因素，而数据信息流则是被动地被调用处理。
8. 相联存储器既可以按地址寻址又可以按内容寻址，也称按内容寻址的存储器

1.2 计算机的性能指标

1. 机器字长（字长）：计算机进行一次整数运算所能处理的位数，一般等于通用寄存器的位数orALU的宽度
2. 数据通路带宽：外部数据总线的宽度
3. 主存容量：字数×字长 2^MAR位数 × MDR位数
4. 运算速度：

   ①吞吐量：系统在单位时间内处理请求的数量

   响应时间：CPU时间（运行一个程序花费的时间）+ 等待时间（磁盘访问等）

   ②CPU时钟周期：CPU工作的**最小**时间单位　**CPU时钟周期数=指令条数×CPI**

   主频（CPU时钟频率）：1/时钟周期

   ③CPI：执行一条指令所需的时钟周期数

   ④CPU执行时间：整个程序，　**=（指令条数 × CPI）/ 主频**

   ⑤IPS：每秒多少条 **=主频/平均CPI =指令条数/CPU执行时间**　MIPS

   ⑥FLOPS：每秒多少次浮点运算　KMGTPEZ
5. 基准程序
6. CPU周期，即机器周期，由多个时钟周期组成
7. IR，MAR，MDR对程序员是透明的，PC，状态寄存器，通用寄存器是汇编程序员可见的

2 数据表示和运算

2.1 数制和编码

1. 二进制B，十进制D，十六机制H、0x
2. 十进制数转换为任意进制数：整数部分---除基取余法，小数部分---乘基取整法
3. 不是每个十进制小数都可以准确的用二进制表示，反之可以，整数也可以
4. 真值 vs 机器数（0正1负）
5. 定点数（小数点位置固定，常规计数） vs 浮点数（科学计数法）
6. 定点数：

①无符号数：无符号位，全是数值位。n位无符号数表示范围：0 ~ 2^n-1。

无符号数只有整数没有小数，unsigned int/long（float ×）

②有符号数：符号位+尾数。整数、小数部分分别保存，小数点分别在最后和符号位之后

（1）原码：尾数表示真值绝对值，符号位0正1负。整数范围 -(2^n-1) ~ 2^n-1，有+0和-0

（2）反码：符号位0和原码相同，符号位1则数值位取反。范围同原码，有+0和-0

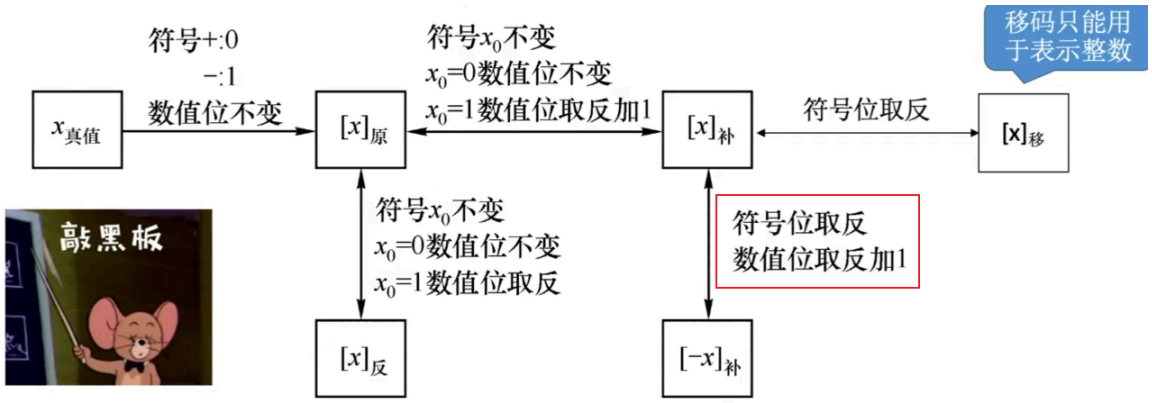
（3）补码：正数同原码，负数=反码+1要进位。整数范围 **-2^n** ~ 2^n-1，仅一个0。$[x]_{补}$=1,0000000表示-2^7。小数会多一个-1

（4）移码：**只能表示整数**。补码符号位取反。范围同补码。仅一个0。方便比较整数大小（按真值递增来的）

行数	机器数	真值(十进制)				
		无符号数	原码	反码	补码	移码
1	0000 0000	0	+0	+0	+0,-0	-128
2	0000 0001	1	+1	+1	+1	-127
3	0000 0010	2	+2	+2	+2	-126
…	…	…	…	…	…	…
126	0111 1101	125	+125	+125	+125	-3
127	0111 1110	126	+126	+126	+126	-2
128	0111 1111	127	+127	+127	+127	-1
129	1000 0000	128	-0	-127	-128	0
130	1000 0001	129	-1	-126	-127	1
131	1000 0010	130	-2	-125	-126	2
…	…	…	…	…	…	…
253	1111 1100	252	-124	-3	-4	124
254	1111 1101	253	-125	-2	-3	125
255	1111 1110	254	-126	-1	-2	126
256	1111 1111	255	-127	-0	-1	127

原码和反码的真值0有两种表示

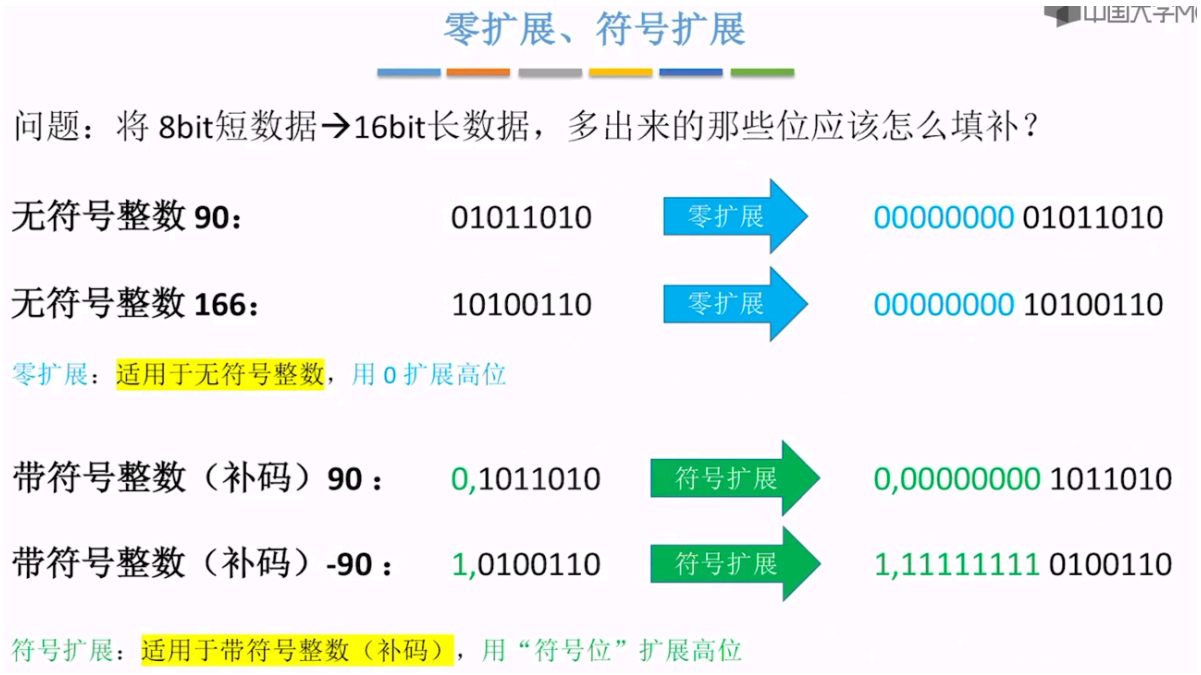
补码和移码的真值0只有一种表示
补码和移码可以多表示一个负数



原码和反码的真值0有两种表示；补码和移码的真值0只有一种表示。若机器字长为n+1位，则：

原码和反码 ——整数表示范围$-(2^n-1) \le x \le 2^n-1$；小数表示范围$-(1-2^{-n}) \le x \le 1-2^{-n}$
补码 ——整数表示范围$-2^n \le x \le 2^n-1$；小数表示范围$-1 \le x \le 1-2^{-n}$
移码 ——整数表示范围$-2^n \le x \le 2^n-1$；移码全0真值最小，移码全1真值最大

7. 模 - 绝对值 = 补码 → 补码把减法换为等价的加法，不需要减法器，运算时符号位参与运算。A-B=A+$B_{补}$



记住：无符号数高位补0，带符号数（补码）高位补符号位

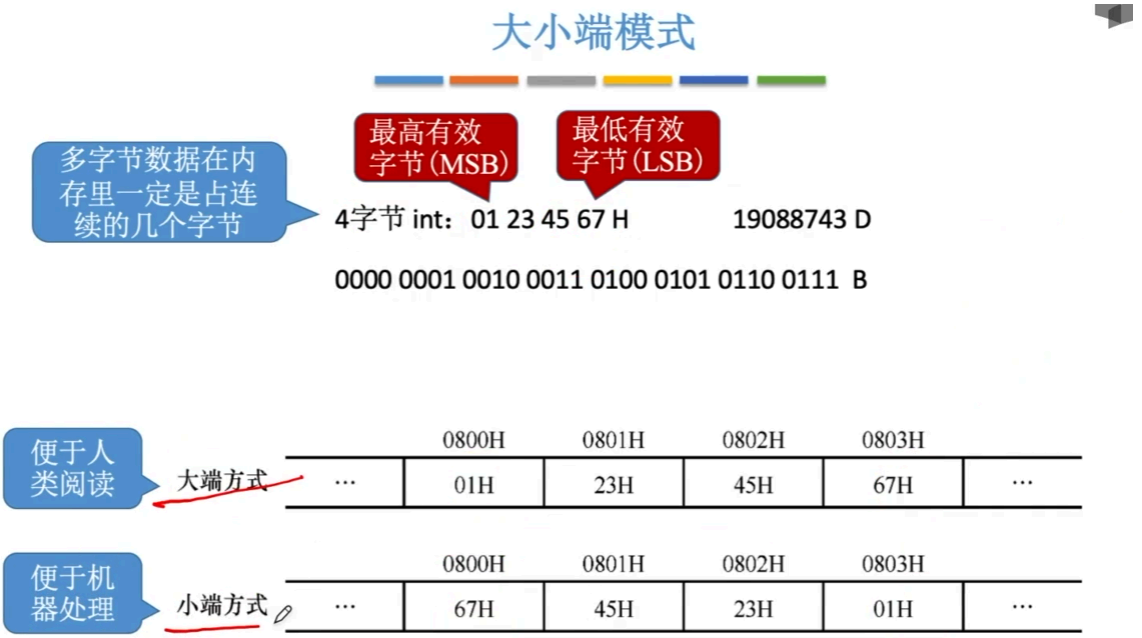
强制类型转换:

① 相同类型有符号数和无符号数之间:机器数不变,改变解释方式

② 长变短:高位截断

③ 短变长:高位符号扩展

int转float就看int写成二进制,看有效位超过24位没,没超过就不丢失精度



小端方式:低的字节67在低地址

大端方式:低的字节67在高地址

由于书写和题目都是左低右高 地址的图,因此大端方式低字节在高地址就在右边,和阅读习惯一样。把图左转90度就是下低上高了

边界对齐

现代计算机通常是按字节编址，即每个字节对应1个地址
通常也支持按字、按半字、按字节寻址。
假设存储字长为32位，则1个字=32bit，半字=16bit。每次访存只能读/写1个字

字节 1	字节 2	字节 3	填充
半字 1		半字 2	
半字 3		填充	
字 1			

访问一个字/半字
都只需一次访存

图 2.10　边界对齐方式

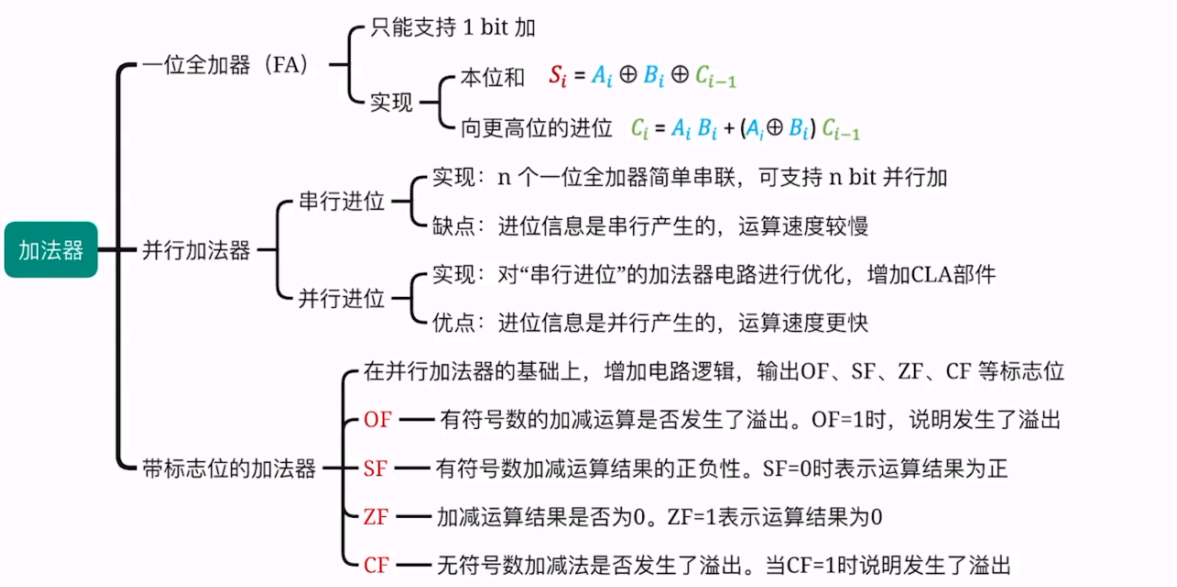
字节 1	字节 2	字节 3	半字 1–1
半字 1–2	半字 2		半字 3–1
半字 3–2	字 1–1		
字 1–2			

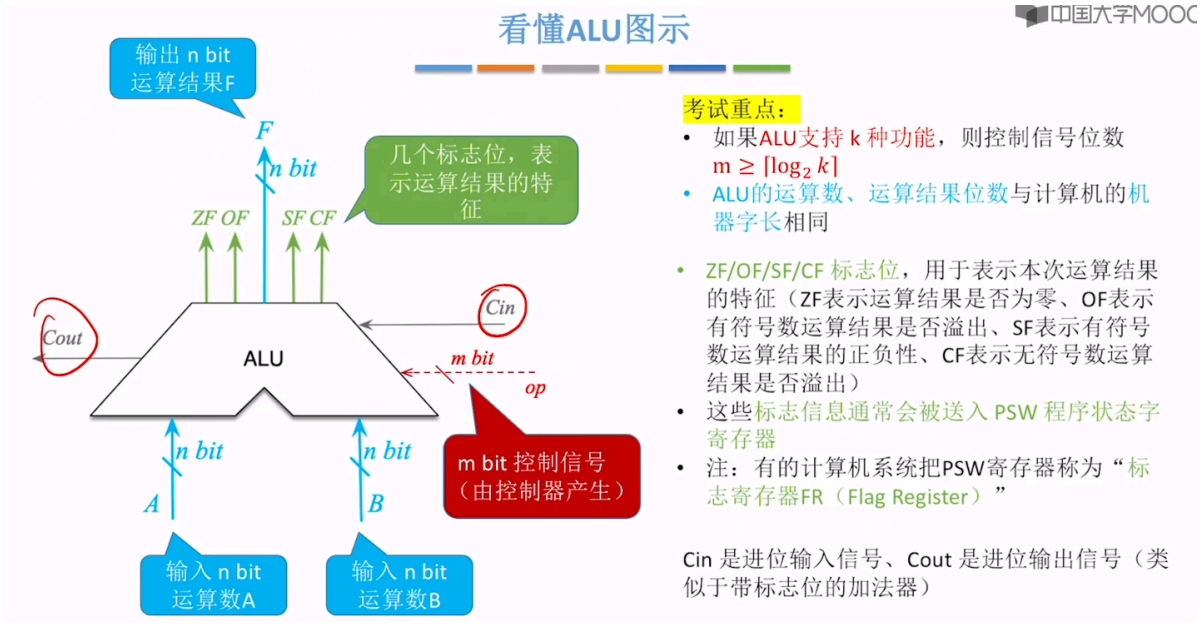
访问一个字/半字
可能要两次访存

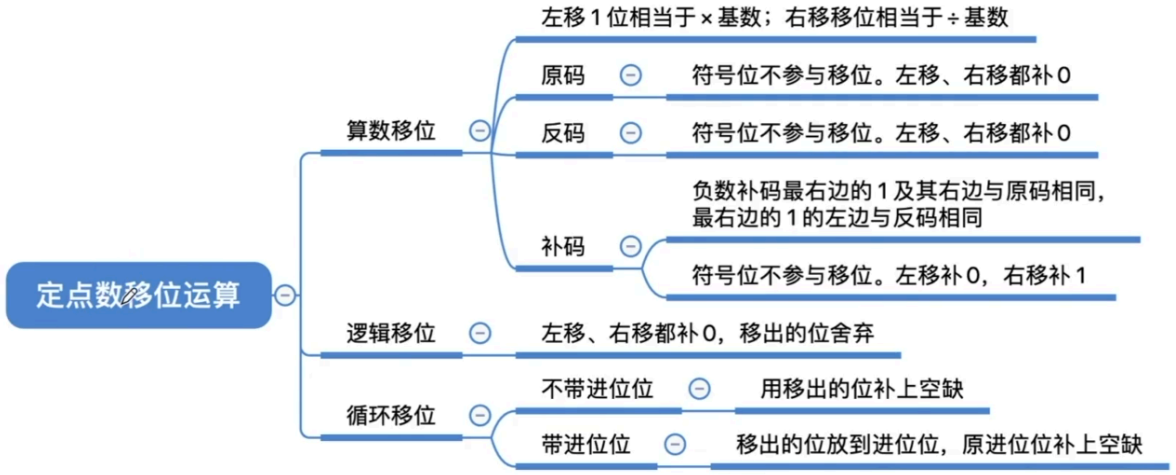
图 2.11　边界不对齐方式

边界对齐：注意结构体的话整个结构体按照内部最大元素对齐

2.2 运算方法和运算电路（定点数）





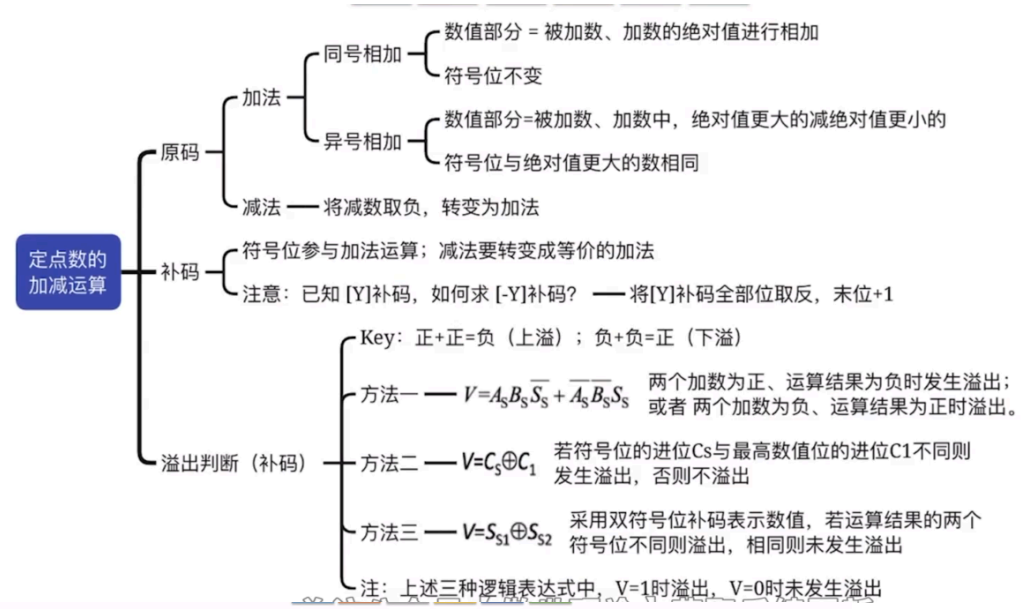


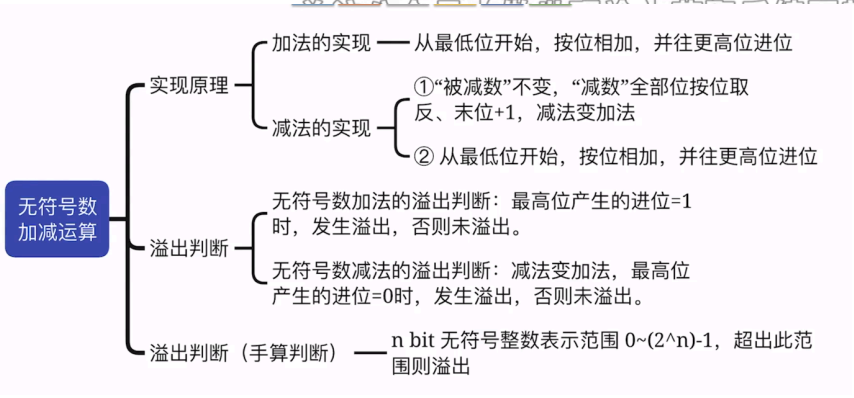
注意：由于原、反、补码位数有限，因此某些时候算数移位不能精确等效乘法、除法

上面不对，算数左移实际上符号位要一起动，然后左移结束后如果符号位改变了，说明溢出了。

算术移位：正数的话原反补都是高低都是0，负数的话：原码高低都是0，**反码高低都是1，补码高1低0。**

然后算术移位是符号位也移动的，左移如果符号位改变了，就是溢出；**右移高位补的符号位是对于补码来说的**，右移最低位1移出会丢失精度

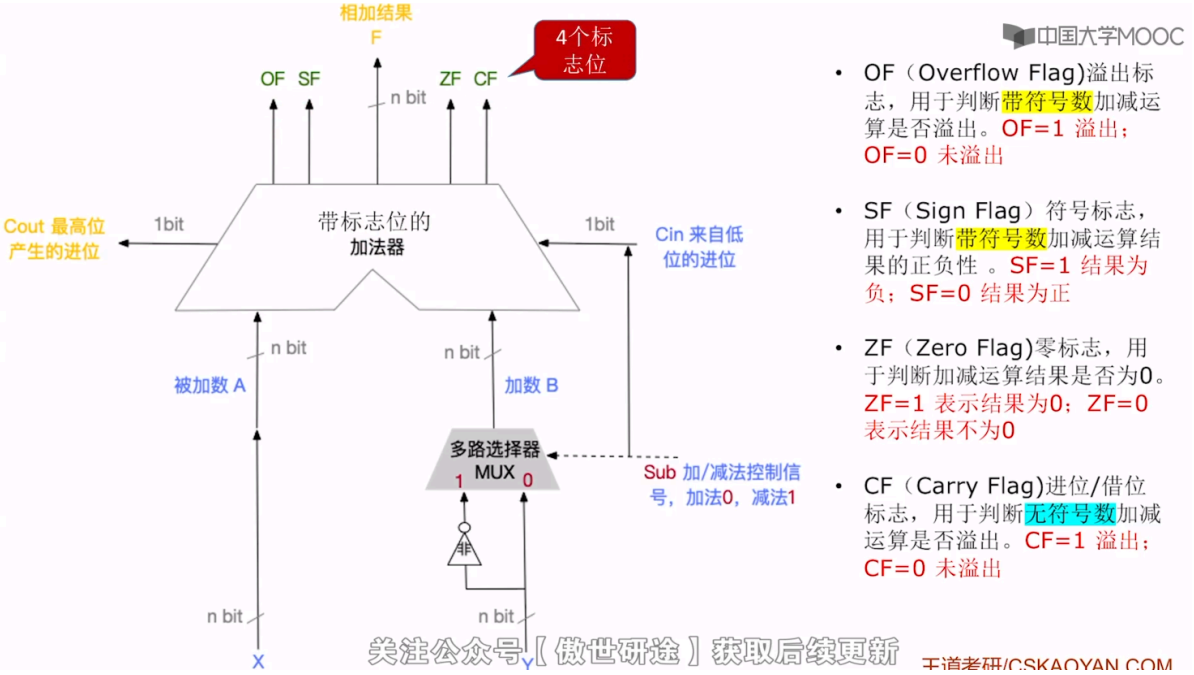




注意无符号数减法是减法变加法，然后进位为0时溢出（模的思想，够减的话换加法会需要减掉一个模，也就是进位为1），这也正是CF溢出判断的原理，Cin 异或 Cout（减法Cin为1，没进位就是Cout为0，异或就是1表示溢出，如下图）

OF：最高数值位进位 异或 符号位进位（如果是减法，转换成加法后再以此判断）

手算判断：OF：正+正变成负 / 负+负变成正 （正+负一定不会溢出） CF：加法最高位进位了 / 减法借位了(也就是算出来小于0)



下面是乘除法：

原码一位乘法（手算模拟）

设机器字长为5位（含1位符号位，n=4），$x=-0.1101$，$y=+0.1011$，采用原码一位乘法求$x\cdot y$

解：$|x|=00.1101$，$|y|=00.1011$，原码一位乘法的求解过程如下。

ACC　通用寄存器　MQ

	(高位部分积)	(低位部分积/乘数)	说明
	00.0000	1011 丢失位	起始情况
+|x|	00.1101		C_4=1, 则+|x|
	00.1101		
右移	00.0110	----- 1101 1	右移部分积和乘数
+|x|	00.1101		C_4=1, 则+|x|
	01.0011		
右移	00.1001	----- 1110 11	右移部分积和乘数
+0	00.0000		C_4=0, 则+0
	00.1001		
右移	00.0100	----- 1111 011	右移部分积和乘数
+|x|	00.1101		C_4=1, 则+|x|
	01.0001		
右移	00.1000	----- 1111 1011	右移部分积和乘数 乘数全部移出

结果的绝对值部分

Tips：
- 乘数的符号位不参与运算，可以省略
- 原码一位乘可以只用单符号位
- 答题时最终结果最好写为原码机器数

原码一位乘法：机器字长n+1，数值部分占n位
符号位通过异或确定；数值部分通过被乘数和乘数绝对值的 n 轮加法、移位完成
根据当前乘数中参与运算的位确定(ACC)加什么。若当前运算位=1，则(ACC)+[|x|]原；若=0，则(ACC)+0。
每轮加法后ACC、MQ的内容统一逻辑右移

符号位 $P_s=x_s\oplus y_s=1\oplus 0=1$，得 $x\cdot y=-0.10001111$。

补码一位乘法（手算模拟）

设机器字长为5位（含1位符号位，n=4），$x=-0.1101$，$y=+0.1011$，采用Booth算法求$x\cdot y$

$[x]_{补}$=11.0011，$[-x]_{补}$=00.1101，$[y]_{补}$=0.1011

	(高位部分积)	(低位部分积/乘数)	说明
	00.0000	0.1011 0 丢失位	起始情况
$+[-x]_{补}$	00.1101	辅助位	Y_4Y_5=10，Y_5-Y_4=-1，则$+[-x]_{补}$
	00.1101		
右移	00.0110	----- 10.1011 0	右移部分积和乘数
+0	00.0000		Y_4Y_5=11，Y_5-Y_4=0，则+0
	00.0110		
右移	00.0011	----- 010.1011 0	右移部分积和乘数
$+[x]_{补}$	11.0011		Y_4Y_5=01，Y_5-Y_4=1，则$+[x]_{补}$
	11.0110		
右移	11.1011	----- 0010.1101 0	右移部分积和乘数
$+[-x]_{补}$	00.1101	原符号位	Y_4Y_5=10，Y_5-Y_4=-1，则$+[-x]_{补}$
	00.1000		
右移	00.0100	----- 00010.1011 0	右移部分积和乘数
$+[x]_{补}$	11.0011		Y_4Y_5=01，Y_5-Y_4=1，则$+[x]_{补}$
	11.0111	构成$[x\cdot y]_{补}$	

正数算数右移　负数算数右移　最后多一次加法

n轮加法、算数右移，加法规则如下：
辅助位 - MQ中最低位 = 1时，(ACC)+$[x]_{补}$
辅助位 - MQ中最低位 = 0时，(ACC)+0
辅助位 - MQ中最低位 = -1时，(ACC)+$[-x]_{补}$

补码的算数右移：
符号位不动，数值位右移，正数右移补0，负数右移补1（符号位是啥就补啥）

$[x\cdot y]_{补}=11.01110001$
即$x\cdot y=-0.10001111$

关注公众号『微世研途』获取后续更新

原码一位乘法：
符号位通过异或确定，数值位由被乘数和乘数的绝对值进行 n 轮加法、移位

每次加法可能 +0 、+[|x|]原

每次移位是“逻辑右移”

乘数的符号位不参与运算

朋友，过两招？

补码一位乘法（Booth算法）：
符号位、数值位都是由被乘数和乘数进行 n 轮加法、移位，最后再多来一次加法

每次加法可能 +0 、+$[x]_{补}$、+$[-x]_{补}$

每次移位是“补码的算数右移”

乘数的符号位参与运算

MQ中最低位 = 1时，(ACC)+[|x|]原
MQ中最低位 = 0时，(ACC)+0

辅助位 - MQ中“最低位” = 1时，(ACC)+$[x]_{补}$
辅助位 - MQ中“最低位” = 0时，(ACC)+0
辅助位 - MQ中“最低位” = -1时，(ACC)+$[-x]_{补}$

原码除法：恢复余数法（手算）

符号位	绝对值

符号位与数值位分开处理

设机器字长为5位（含1位符号位，$n=4$），$x=0.1011$，$y=0.1101$，采用原码恢复余数法求x/y

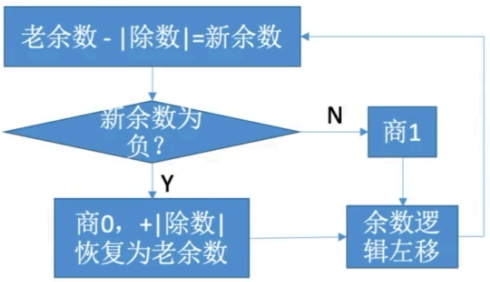
$|x|=0.1011$，$|y|=0.1101$，$[|y|]_{补}=0.1101$，$[-|y|]_{补}=1.0011$

被除数/余数		商
	0.1011	
$+[-\|y\|]_{补}$	1.0011	
	1.1110	0
$+[\|y\|]_{补}$	0.1101	
	0.1011	
左移	1.0110	
$+[-\|y\|]_{补}$	1.0011	
	0.1001	01
左移	1.0010	
$+[-\|y\|]_{补}$	1.0011	
	0.0101	011

余数为负，就要商0，并恢复余数

逻辑左移

余数为正，就要商1，不用恢复余数



原码除法：加减交替法

又名：不恢复余数法

符号位与数值位分开处理

设机器字长为5位（含1位符号位，$n=4$），$x=0.1011$，$y=0.1101$，采用原码加减交替除法求x/y

$|x|=0.1011$，$|y|=0.1101$，$[|y|]_{补}=0.1101$，$[-|y|]_{补}=1.0011$

若余数为负，则可直接商0，让余数左移1位再加上|除数|，得到下一个新余数

若余数为正，则商1，让余数左移1位再减去|除数|，得到下一个新余数

被除数/余数		商	ACC	MQ
	0.1011		01011	00000
$+[-\|y\|]_{补}$	1.0011			
	1.1110	0	11110	00000
左移	1.1100		11100	00000
$+[\|y\|]_{补}$	0.1101			
	0.1001	01	01001	00001
左移	1.0010		10010	00010
$+[-\|y\|]_{补}$	1.0011			
	0.0101	011	00101	00011
左移	0.1010		01010	00110
$+[-\|y\|]_{补}$	1.0011			
	1.1101	0110	11101	00110
左移	1.1010		11010	01100
$+[\|y\|]_{补}$	0.1101			
	0.0111	0110	00111	01100

若余数为负，需商0，并$+[|y|]_{补}$得到正确余数

$Q_s=x_s \oplus y_s=0 \oplus 0=0$

得$x/y=+0.1101$

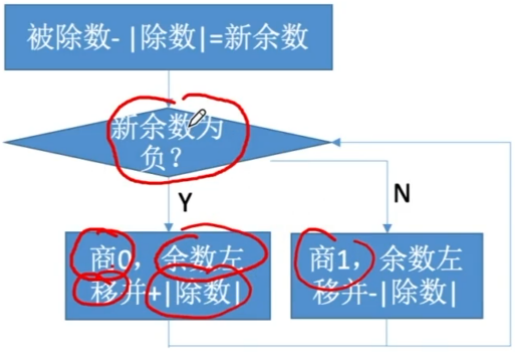
余0.0111×2^{-4}

注：余数的正负性与商相同

恢复余数法：当余数为负时商0，并+|除数|，再左移，再-|除数|

加减交替法：当余数为负时商0，并左移，再+|除数|

关注公众号【做世研选】获取后续更新
王道考研/CSKAOYAN.COM



补码除法：加减交替法

设机器字长为5位（含1位符号位，n=4），x=+0.1000，y=-0.1011，采用补码加减交替除法求x/y

$[x]_{补}$=00.1000，$[y]_{补}$=11.0101，$[-y]_{补}$=00.1011

被除数/余数	ACC	MQ
00.1000	001000	00000
$+[y]_{补}$ 11.0101		
11.1101	111101	00001
逻辑左移 左移 11.1010	111010	00010
$+[-y]_{补}$ 00.1011		
00.0101	000101	00010
左移 00.1010	001010	00100
$+[y]_{补}$ 11.0101		
11.1111	111111	00101
左移 11.1110	111110	01010
$+[-y]_{补}$ 00.1011		
00.1001	001001	01010
左移 01.0010	010010	10100
$+[y]_{补}$ 11.0101		
00.0111	000111	10101

精度误差不超过 2^{-n}

末位商恒置1

补码除法：
- 符号位参与运算
- 被除数/余数、除数采用双符号位

被除数和除数同号，则被除数减去除数；
异号则被除数加上除数。

余数和除数同号，商1，余数左移一位减去除数；
余数和除数异号，商0，余数左移一位加上除数。
重复n次

除法类型	符号位参与运算	加减次数	移位		上商、加减原则	说明
			方向	次数		
原码加减交替法	否	N+1或N+2	左	N	余数的正负	若最终余数为负，需恢复余数
补码加减交替法	是	N+1	左	N	余数和除数是否同号	商末位恒置1

模4补码（双符号位）存储时只需要存储一个符号位，只用把两个模4补码得数送往ALU进行加减运算时，才把每个数的符号位的值同时送到ALU的双符号位，即只在ALU中采用双符号位，正溢出01，负溢出10

如果同时让判断一个运算的OF，CF标志位，则分别当做有符号数的运算和无符号数的运算分别判断

2.3 浮点数的表示与运算（浮点数）

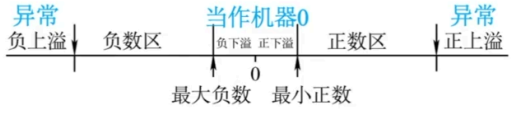
规格化浮点数的特点

规格化的原码尾数，最高数值位一定是1

1. 用原码表示的尾数进行规格化：
正数为0.1××…×的形式，其最大值表示为0.11…1；最小值表示为0.10…0。
尾数的表示范围为$1/2 \le M \le (1-2^{-n})$。
负数为1.1××…×的形式，其最大值表示为1.10…0；最小值表示为1.11…1。
尾数的表示范围为$-(1-2^{-n}) \le M \le -1/2$。

规格化的补码尾数，符号位与最高数值位一定相反

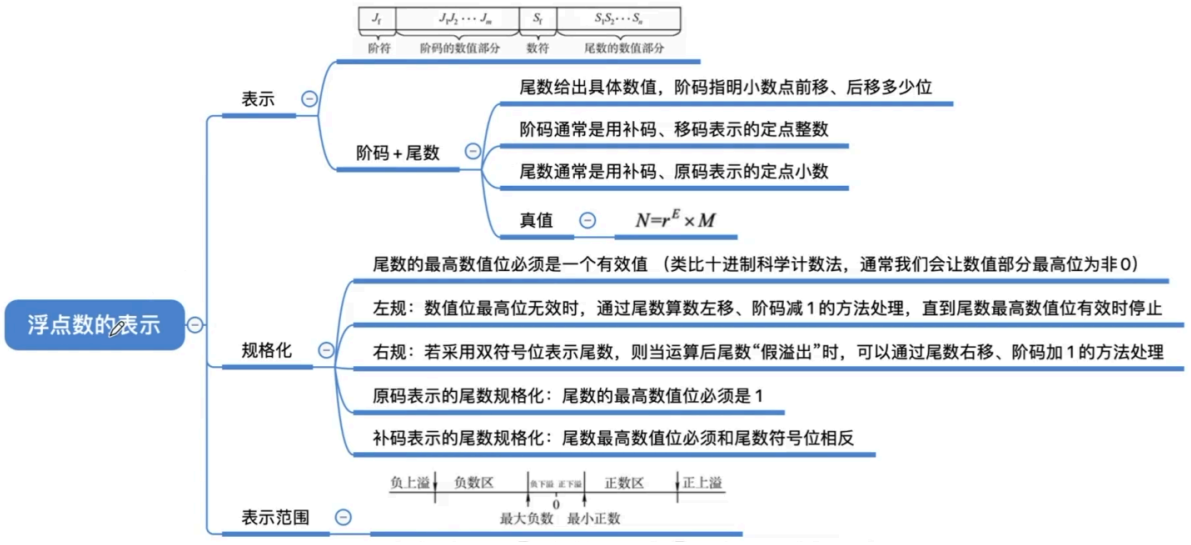
2. 用补码表示的尾数进行规格化：
正数为0.1××…×的形式，其最大值表示为0.11…1；最小值表示为0.10…0。
尾数的表示范围为$1/2 \le M \le (1-2^{-n})$。
负数为1.0××…×的形式，其最大值表示为1.01…1；最小值表示为1.00…0。
尾数的表示范围为$-1 \le M \le -(1/2+2^{-n})$。



eg：若某浮点数的阶码、尾数用补码表示，共4+8位：0.110；1.1110100 如何规格化？

注：补码算数左移，低位补0；补码算数右移，高位补1。

上面已经删了



移码

偏置值一般取 2^{n-1}，此时移码=补码符号位取反

真值(十进制)	补码	移码
-128	1000 0000	0000 0000
-127	1000 0001	0000 0001
-126	1000 0010	0000 0010
…	…	…
-3	1111 1101	0111 1101
-2	1111 1110	0111 1110
-1	1111 1111	0111 1111
0	0000 0000	1000 0000
1	0000 0001	1000 0001
2	0000 0010	1000 0010
3	0000 0011	1000 0011
…	…	…
124	0111 1100	1111 1100
125	0111 1101	1111 1101
126	0111 1110	1111 1110
127	0111 1111	1111 1111

真值增大

移码的定义：移码=真值+偏置值

此处8位移码的偏置值=128D=1000 0000B，即2^{n-1}

真值 -127 = -1111111B
移码 = -1111111 + 10000000 = 0000 0001

真值 -3= -11B
移码 = -11 + 10000000 = 0111 1101

真值 +0 = +0
移码 = +0 + 10000000 = 1000 0000

真值 +3 = +11B
移码 = +11 + 10000000 = 1000 0011

真值 +127 = +1111111B
移码 = +1111111 + 10000000 = 1111 1111

移码

偏置值 $=2^{n-1}$

偏置值$=2^{n-1}-1$

真值(十进制)	补码	移码	移码
-128	1000 0000	0000 0000	1111 1111
-127	1000 0001	0000 0001	0000 0000
-126	1000 0010	0000 0010	0000 0001
…	…	…	…
-3	1111 1101	0111 1101	0111 1100
-2	1111 1110	0111 1110	0111 1101
-1	1111 1111	0111 1111	0111 1110
0	0000 0000	1000 0000	0111 1111
1	0000 0001	1000 0001	1000 0000
2	0000 0010	1000 0010	1000 0001
3	0000 0011	1000 0011	1000 0010
…	…	…	…
124	0111 1100	1111 1100	1111 1011
125	0111 1101	1111 1101	1111 1100
126	0111 1110	1111 1110	1111 1101
127	0111 1111	1111 1111	1111 1110

无符号数255

无符号数1

无符号数-1

真值增大

偏置值可以取其他值

移码的定义：移码=真值+偏置值

令偏置值=127D=0111 1111B，即$2^{n-1}-1$

真值 -128 = -1000 0000B
移码 = -1000 0000 + 01111111 = 1111 1111

真值 -127 = -111 1111B
移码 = -111 1111 + 01111111 = 0000 0000

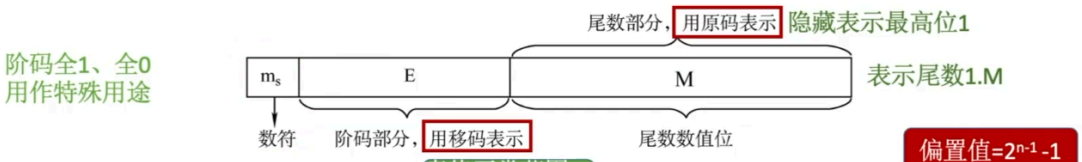
真值 -126 = -111 1110B
移码 = -111 1110 + 01111111 = 0000 0001

真值 +0 = +0
移码 = +0 + 01111111 = 0111 1111

真值 +127 = +1111111B
移码 = +111 1111 + 01111111 = 1111 1110

关注公众号【 】获取后续更新
王道考研/CSKAOYAN.COM

IEEE 754标准



偏置值=$2^{n-1}-1$

真值正常范围：-126~127

类型	数符	阶码	尾数数值	总位数	偏置值（十六进制）	偏置值（十进制）
单精度浮点型 float	短浮点数	1	8	23	32	7FH	127
双精度浮点型 double	长浮点数	1	11	52	64	3FFH	1023
long double	临时浮点数	1	15	64	80	3FFFH	16383

float 1000 0001 1000 1010 0101 0000 1000 0000

double 1000 0001 1100 0010 0101 0000 1000 0000 0000 0000 0001 1111 0000 0000 0000 0000

规格化的短浮点数的真值为：$(-1)^s \times 1.M \times 2^{E-127}$

规格化长浮点数的真值为：$(-1)^s \times 1.M \times 2^{E-1023}$

阶码真值=移码-偏移量

例：将十进制数 -0.75 转换为 IEEE 754 的单精度浮点数格式表示。

$(-0.75)_{10} = (-0.11)_2 = (-1.1)_2 \times 2^{-1}$

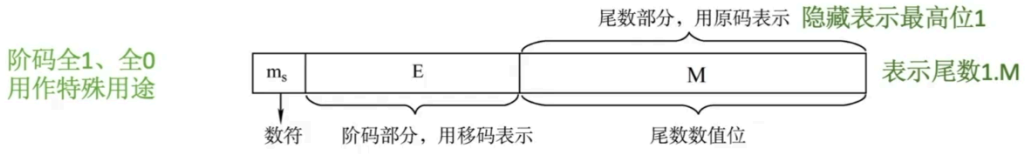
数符 = 1
尾数部分 = .1000000.....（隐含最高位1）
阶码真值 = -1
单精度浮点型偏移量 = 127D
移码 = 阶码真值+偏移量 = -1 + 111 1111 = 0111 1110（凑足8位）

→ 1 01111110 10000000000000000000000

例：IEEE 754 的单精度浮点数 C0 A0 00 00 H 的值时多少。

C0 A0 00 00 H → 1100 0000 1010 0000 0000 0000 0000 0000

数符 = 1 → 是个负数
尾数部分 = .0100....（隐含最高位1）→ 尾数真值 = $(1.01)_2$
移码 = 10000001，若看作无符号数= 129D
单精度浮点型偏移量 = 127D
阶码真值= 移码 - 偏移量 = 1000 0001 – 111 1111 = $(0000\ 0010)_2 = (2)_{10}$
→ 浮点数真值 = $(-1.01)_2 \times 2^2 = -1.25 \times 2^2 = -5.0$



IEEE 754 单精度浮点型能表示的最小绝对值、最大绝对值是多少？

若要表示的数绝对值还要更小，怎么办？

最小绝对值：尾数全为0，阶码真值最小-126，对应移码机器数 0000 0001
此时整体的真值为 $(1.0)_2 \times 2^{-126}$

只有 $1 \le E \le 254$时，真值 $= (-1)^s \times 1.M \times 2^{E-127}$

隐含最高位变为0

阶码真值固定视为 -126

当阶码E全为0，尾数M不全为0时，表示非规格化小数 $\pm(0.xx...x)_2 \times 2^{-126}$

当阶码E全为0，尾数M全为0时，表示真值 ±0

当阶码E全为1，尾数M全为0时，表示无穷大 ±∞

当阶码E全为1，尾数M不全为0时，表示非数值 "NaN"（Not a Number）

如：0/0、∞-∞ 等非法运算的结果就是 NaN

注意阶码为0尾数不为0时，隐含的是0，**阶码还是-126**，而不是-127

记：尾数是0，就表示0/无穷，尾数不是0，那么最小就是非规格化小数，最大就是0/0不是数了

浮点数的加减运算

例：已知十进制数$X=-5/256$、$Y=+59/1024$，按机器补码浮点运算规则计算$X-Y$，结果用二进制表示，浮点数格式如下：阶符取2位，阶码取3位，数符取2位，尾数取9位

用补码表示阶码和尾数

0. 转换格式

5D = 101B，$1/256 = 2^{-8}$ → $X = -101 \times 2^{-8} = -0.101 \times 2^{-5} = -0.101 \times 2^{-101}$

59D = 111011B，$1/1024 = 2^{-10}$ → $Y = +111011 \times 2^{-10} = +0.111011 \times 2^{-4} = +0.111011 \times 2^{-100}$

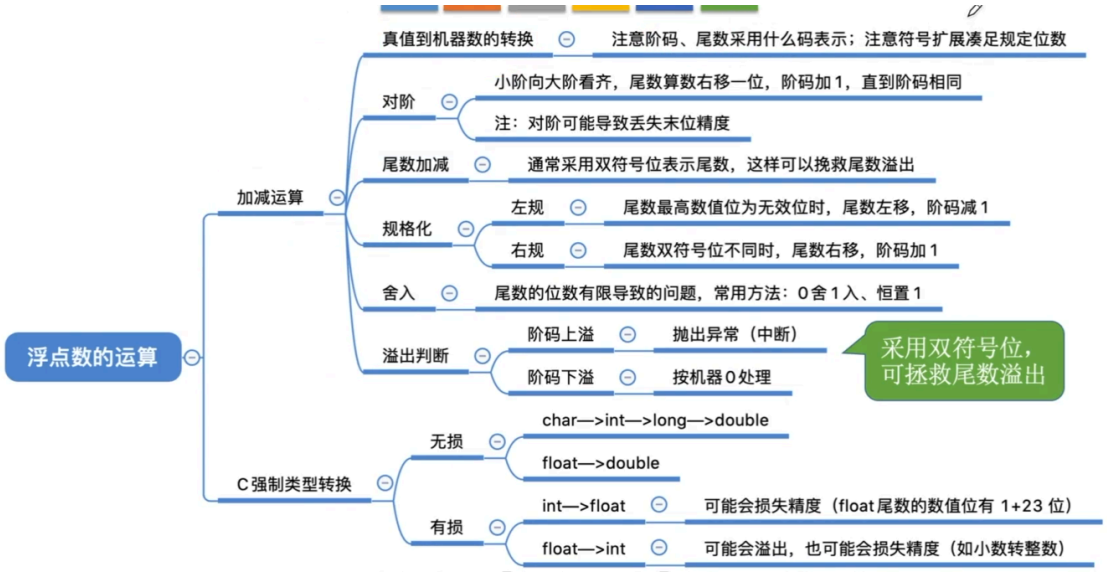
X：11011,11.011000000　　Y：11100,00.111011000

浮点数加减运算步骤：

1. 对阶 使两个数的阶码相等，小阶向大阶看齐，尾数每右移一位，阶码加1
   ① 求阶差：$[\Delta E]_{补}=11011+00100=11111$，知$\Delta E=-1$
   ② 对阶：X：11011,11.011000000 → 11100,11.101100000　　$X = -0.0101 \times 2^{-100}$

2. 尾数加减 -Y：11100,11.000101000　　11.101100000　　X-Y
   X-Y：11100, 10.110001000　　+ 11.000101000　　$= (-0.0101 \times 2^{-100}) - (+0.111011 \times 2^{-100})$
   10.110001000　　$= (-0.0101-0.111011) \times 2^{-100}$
3. 规格化　　$= -1.001111 \times 2^{-100}$
   X-Y：11100, 10.110001000 → 11101,11.011000100　　$= -0.1001111 \times 2^{-011}$
4. 舍入　无舍入
5. 判溢出　常阶码，无溢出，结果真值为$2^{-3} \times (-0.1001111)_2$

注意：给定的这种，小数点是在最高位之前的，也就是都是0.多，而不是1.多



舍入是浮点数的概念，定点数没有舍入的概念。浮点数舍入有两种情况：对阶、右规格化。舍入不一定产生误差，如向下舍入11.00到11.0时是没有误差的

规格化IEEE 754浮点数尾数部分的数值范围为[1,2)。如果给的是十进制的数，就先写成±1.xxxxx * 2^n的形式，再看要不要写回之类的。

32位机：int 32，long 32，longlong 64，float 32，double 64

原码：正数规格化形式0.1.....，负数规格化形式1.1.....，小数点前表示符号位，若基数为4，则用2位表示，则原码表示的规格化小数点后2位不全为0即可（基数为2的话，就是小数点后必然是1，IEEE的话是省了）

补码：正数0.1.....，负数1.0.....

3 存储系统

3.1 存储器概述

1. 磁表面存储器（非易失，磁盘、磁带），磁芯存储器，半导体存储器（MOS型、双极型），光存储器（非易失，光盘CD-ROM）
2. ①随机存储器 RAM：随机存取，主存和Cache，破坏性读出SRAM，非破坏性读出DRAM

   ②只读存储器 ROM：断电不丢失，随机存取，和RAM共同组成主存

   ③串行访问存储器：顺序存取存储器 SAM（磁带），直接存取存储器 DAM（磁盘、光盘，不是随机）

   ④相联存储器 CAM：按内容和地址相结合访问，快表TLB
3. 性能指标：

   ①存储容量：存储字数×字长（1.2的主存容量）

   ②单位成本：总成本/总容量

   ③存储速度：**主存带宽Bm，即数据传输速率 = 数据宽度（即字长）/存取周期**

   存取周期Tm = 存取时间Ta + 恢复时间　　存取时间=总线(传输)周期
4. Cache-主存层主要解决：CPU和主存速度不匹配的问题（主存-Cache间的数据调动硬件自动完成，对所有程序员透明）

   主存-辅存层主要解决：存储系统的容纳量问题（硬件与OS共同完成，对应用程序员透明）

3.2 主存储器

1. RAM：易失

   ①SRAM：静态，Cache，双稳态触发器，单译码法，非破坏性读出

   ②DRAM：动态，主存，栅极电容，**双译码法（地址引脚复用）**，破坏性读出 → 定时刷新（独立完成，对CPU透明），读后再生：

   集中刷新（访问死区，每个刷新周期固定）、分散刷新（延长存取周期）、异步刷新（按行间隔刷新）

   → SDRAM（同步DRAM）：支持突发传输方式（传送多个连续单元），送到行缓冲器（SRAM实现）大小=列数×位平面数

   256k*8b则：
   2^9 * 2^9 * 8
   行数列数都为 9
   位平面数为 8
   位平面大小 256k
   行缓冲 9 * 8
2. ROM：非易失

   ①MROM 掩模式ROM：出厂即写固定

   ②PROM 一次可编程ROM：用户写一次

   ③EPROM 可擦除可编程ROM：可读可写（次数有限，写入慢），UVEPROM：紫外线，所有；EEPROM：电擦除，特定字

④Flash 闪存：快速擦除重写，手机辅存、U盘

⑤SSD 固态硬盘： 基于闪存，控制单元+存储单元（Flash芯片）

3. 多模块存储器：空间并行技术

①单体多字存储器：每个存储单元存m个字，总线宽度也m个字，一次读出m条指令

②多体并行存储器：相同容量，独立地址、数据寄存器和控制电路，可并行可交叉

(1) 高位交叉编制（顺序方式）：（体号，块内地址）顺序存储器，单纯是扩容，还是串行存取

(2) 低位交叉编址（交叉方式）：（块内地址，体号）交叉存储器，**轮流启动方式**（访存冲突）or同时启动方式

轮流启动方式：每个模块一次读写位数 = 数据总线位数

交叉模块数m ≥ 模块存取周期T / 总线周期r（总线时间、总线传输周期）

连续存取m个字，需要时间t = T + (m-1) / r

同时启动方式：所有模块一次并行读写总位数 = 数据总线位数，同时启动（如：数据总线64位，4个模块，每个模块一次读写16位）

4. 刷新时构成存储器的所有芯片同时按行刷新

3.3 主存储器与CPU的连接

1. 位扩展法（存储位数），字扩展法（存储字数量，即扩地址，用译码器，最高几位区分芯片），字位同时扩展法
2. CPU访问存储单元：先片选（①线选法：一位对一个片、不连续；②译码片选法：n位对2^n个√），后字选

3.4 外部存储器

1. 磁盘存储器 = 磁盘驱动器 + 磁盘控制器（驱动器与主机的接口） + 盘片
2. 磁盘高速缓存：在内存中开辟的区域，缓冲将被送到磁盘上的数据（联系操作系统中说的页面链表）
3. 面密度 = 道密度（单位长度的磁道数） × 位密度（磁道单位长度的bit数）

非格式化容量 = 记录面数 × 柱面数 × 每条磁道的磁化单元数

格式化容量 = 记录面数 × 柱面数 × 每道扇区数 × 每个扇区的容量 （变小）

存取时间 = 寻道时间 + 旋转延迟时间 + 传输时间 （+磁盘控制器延迟）

4. 磁盘地址：驱动器号 + 柱面号（磁道号）+ 盘面号（磁头号）+ 扇区号
5. RAID 独立冗余磁盘阵列：

RAID0：类似低位交叉编址，条带化技术，无冗余无容错

RAID1：镜像，纯copy

RAID2：海明码纠错（4bit信息+3bit校验）

RAID3-5：奇偶校验

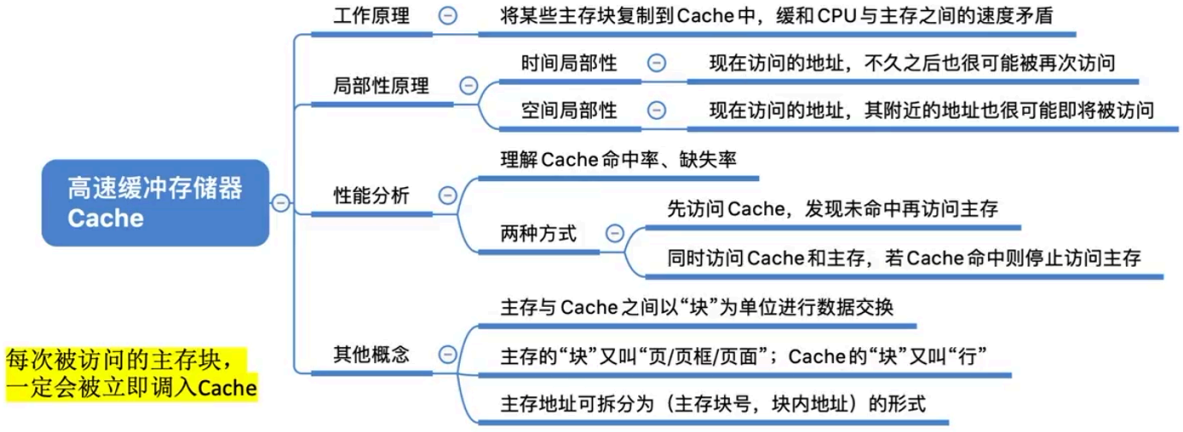
条带化：一种将数据分片，同时读写至不同磁盘，提高读写速度的技术，优点是读写速度快，缺点是没有冗余，如果其中一块磁盘损坏，数据就会丢失。因此条带化不能提高RAID可靠性

6. 磁盘存储器采用直接存储器存取DMA方式进行输入/输出，按批处理方式进行一个数据块的读/写

3.5 高速缓冲存储器

注意真题中：Cache块大小 = 内存块大小，内存中是分页、页框，**内存中一个页能放好多个内存块**

字 是Cache和CPU之间交换单位；**内存块** 是Cache和主存之间交换单位；**页** 是主存和磁盘之间交换单位



- 如何区分 Cache 与 主存 的数据块对应关系？——Cache和主存的映射方式
- Cache 很小，主存很大。如果Cache满了怎么办？——替换算法
- CPU修改了Cache中的数据副本，如何确保主存中数据母本的一致性？——Cache写策略

一手上岸公众号：研途小时

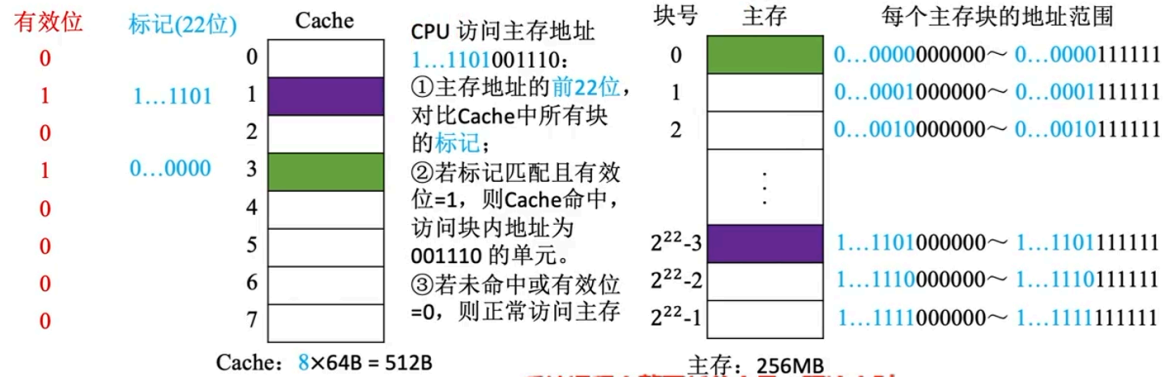
“全相联映射”如何访存？

假设某个计算机的主存地址空间大小为256MB，按字节编址，其数据Cache有8个Cache行，行长为64B。

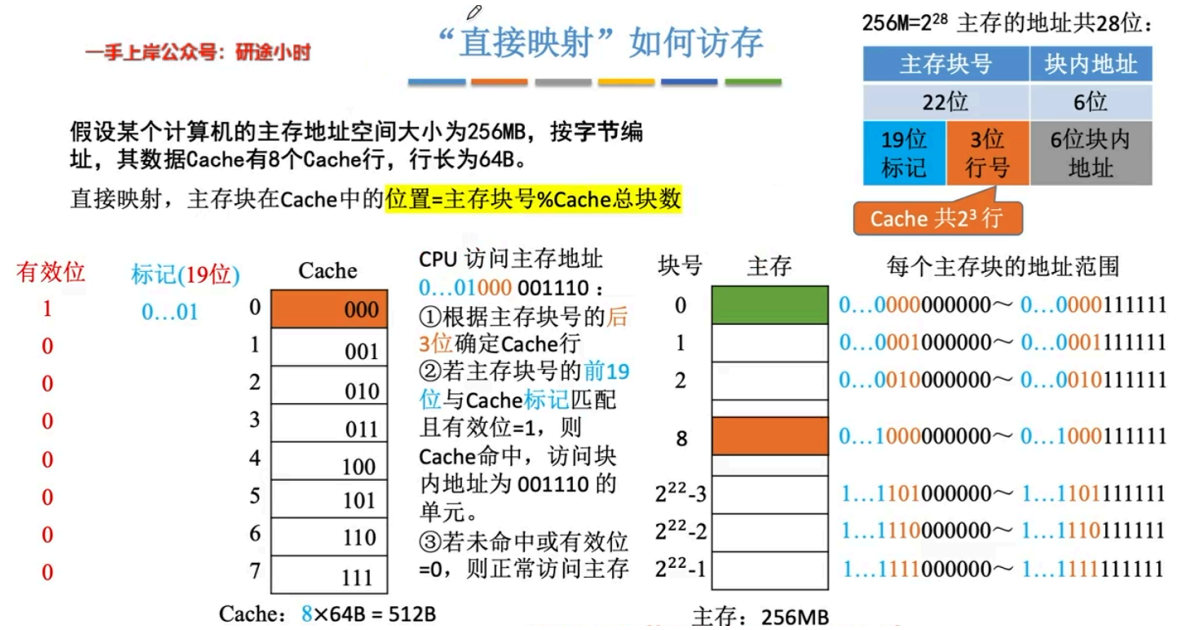
即Cache块，与主存块的大小相等

256M=2^{28} 主存的地址共28位：

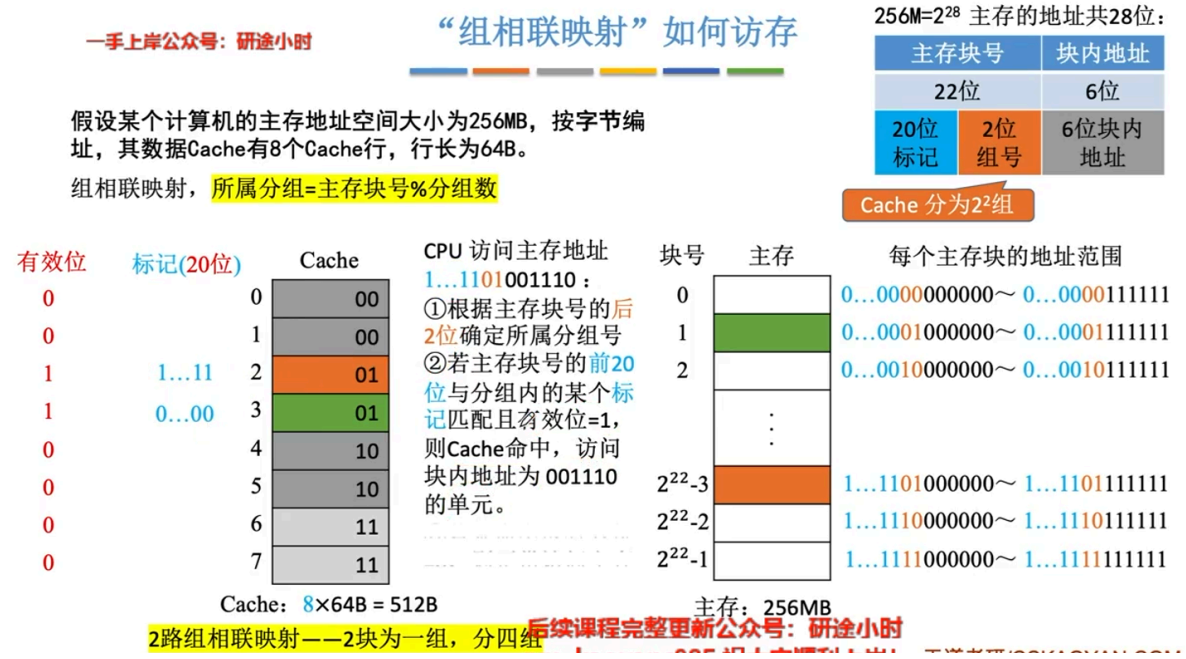
主存块号	块内地址
22位	6位



直接根据标记挨个对比所有Cache块



先确定对应的Cache行，即固定的是哪个位置，再对比标记看放的是不是要找的



先根据组号确定是哪个组，在组里挨个比较看有没对上的。k路组相联就是k个Cache行分成1组，不是所有Cache行总共分成k组

1. Cache和主存的映射方式：

   ①直接映射：（标记，Cache行号，块内地址）主存中每一块对应唯一位置，无条件置换不用置换算法，标记与有效位用来比较

   ②全相联映射：（标记，块内地址）主存每一块可放任何位置，标记指出在主存哪一块，比较速度慢，每个Cache行一个比较器

   ③组相联映射：（标记，组号，块内地址）r路组相联，就有r个比较器只用管一组

   Cache总容量 = 存储容量 + 标记阵列容量（标记项=有效位+脏位+替换算法位+标记位）=（四个位+存储位数）× Cache行数

   （存储容量就是原本一个块的大小，只是需要给每个块加几个各种位，构成一整块Cache的实际位数）

   而主存地址位数就按上面分为标记位+（号）+块内地址

比较器位数就是标记位数

数据位就是cache要存储的数据，对于1B的数据，就是需要8位来存储，1位来表示

同理对于32位的数据，就是32位来存储，5位来表示，平时我们碰到的题目让我们表示的居多，所以容易混淆**表示（定位）**和**存储**的两种概念，给出一块的大小XB,可以得到主存块内地址X位，也可以得到Cache数据位要8X位

因为主存的位数中存放的是地址，一个地址再从主存对应地址取数，算容量时候也是2^地址位数 × 每个地址的数据位数，而在Cache里是不一样的，标记项中的那些位是按bit来的，数据位也是按bit来的，是把对应的所有数据存了过来，而不只是存的地址。

再次总结：

先划分主存地址：如32位，告诉了主存块大小64B（=Cache块大小，是所含数据的量），按字节编址，那么一块就占6位（6位对应64个地址，对应64B的数据），低6位就是块内地址，然后看是什么映射方式：

如直接映射，那么地址是（标记，Cache行号，块内地址）这样的分法，告诉Cache数据区大小64KB（是不含标记项的），则一共64KB/64B=1K个Cache行/块，占10位，剩下标记位16位（16,10,6），那么Cache行位数就是16位标记位+1位有效位+**64×8**位数据位（若是回写策略，再加一位脏位）

如8路组相联映射，那么地址是（标记，组号，块内地址）这样的分法，告诉Cache数据区大小64KB，那么就有64KB/64B=1K个Cache块（Cache块数=Cache行数）8块一组，一共1K/8=128个组，组号占7位，则剩标记位19位（19,7,6），可以得到比较器有8个（几路就是几个），比较器位数是19位（和标记位一样，因为是用来比较是不是要找的块），其他和上面一样

CPU访问时，就是先看组号找到，然后看前19位对照相同且有效位为1，就是找的Cache行，在行中根据块内地址，就是要找的数据

2. 替换算法：

   ①随机算法 RAND：随机

   ②先进先出算法 FIFO

   ③近期最少使用算法 LRU：堆栈类算法，**每个Cache行/组一个计数器（LRU替换位）**命中清零且比其低的加一，比如：如果2路组相联，那么LRU替换位为1位！！每组就2个块，每次替换都是2选1；8路组相联就是3位；一定不要弄混，**分组的话：一共mn块，m路组相联，一共分成n组，那么替换位是$\log_2 m$，而不是$\log_2 n$！**，另外，比较器个数是m

   ④最不经常使用算法 LFU

3. 一致性问题：写命中：　　+　写不命中：

   ①全写法（直写法）：同时写入Cache+主存，用写缓冲 SRAM　+　非写分配法：只更新主存不调入（读未命中才调入）（主存为主

   ②回写法：只写入Cache，替换出才写主存，用脏位（修改位）　+　写分配法：更新主存后也调入Cache　　（Cache为主

   不对不对不对，写分配法应该是先调入Cache再写√（此时主存还没写）

4. CPU -------- L1 Cache（分离指令Cache和数据Cache）----(①各级之间)---- L2 Cache ----(②和主存之间)---- DRAM

3.6 虚拟存储器

1. 虚拟存储机制：全相联映射，回写法
2. TLB 快表：SRAM实现，全相联or组相联，速度快来源于硬件本身，不依赖搜索算法查找
3. Cache主要解决系统速度，而虚拟存储器确实为了解决主存容量
4. **Cache全由硬件实现，是硬件存储器，对所有程序员透明；缺失由硬件完成，从主存读数据**

   虚拟存储器由OS和硬件共同实现，是逻辑上的存储器，对系统程序员不透明，对应用程序员透明；**缺页由软件完成**，TLB硬软都行，是从辅存读数据
5. Cache不命中时主存能和CPU直接通信，同时将数据调入Cache（TLB也是），虚拟存储器系统不命中时，只能先由硬盘调入主存，而不能直接和CPU通信
6. 虚拟地址 ------(先快表再内存)------ 物理地址 -------（找Cache）------ Cache找数据

   Page缺失则TLB和Cache 一定缺失（都缺页不在内存了，说明根本没从磁盘调入呢）
7. 缺页处理完成后回到发生缺页的指令继续执行（重新执行也对）

4 指令系统

4.1 指令系统

1. 指令字长：一条指令的总长度，可能会变，注意按字节编址下，指令字长**一定是8的倍数，17 → 24**

   机器字长：CPU进行一次整数运算所能处理的二进制数据的位数（通常和ALU直接相关

   存储字长：一个存储单元中的二进制代码位数（=MDR位数）
2. 零地址指令：OP，空操作、停机、关中断，仅用在堆栈计算机。堆栈指令的访存次数取决于采用的是软堆栈还是硬堆栈：

   软堆栈：堆栈区由内存实现，双目运算要4次访存：取指、取1、取2、存

   硬堆栈：堆栈区由寄存器实现，只需取指1次访存 RISC

   一地址指令：OP(A1)→A1，3次（单操作数，读+写）/2次（双操作数，隐含ACC，写到ACC）访存

   二地址指令：(A1)OP(A2)→A1，4次访存（取指-读A1-读A2-写A1）

   三地址指令：(A1)OP(A2)→A3，4次访存

   四地址指令：(A1)OP(A2)→A3，A4下条指令地址，4次访存
3. 扩展操作码 = 定长指令字结构（指令长度固定）+ 可变长操作码，保留一位留扩展类似哈夫曼

   每个地址码的长度不一定一样，不一定平分，如寄存器直接寻址的话，通用寄存器n个，则只需要$\log_2 n$位来表示

学上岸课程公众号：研途小助手

扩展操作码

扩展操作码举例

设指令字长固定为16位，试设计一套指令系统满足：

a) 有15条三地址指令
共2^4=16种状态
留出16-15=1种
b) 有12条二地址指令
共1 ×2^4=16种
留出16-12=4种
c) 有62条一地址指令
共4 ×2^4=64种
留出64-62=2种
d) 有32条零地址指令
共2 ×2^4=32种

	0000 - 1110	A_1	A_2	A_3
1111 XXXX XXXX XXXX	1111	0000 - 1011	A_1	A_2
1111 11XX XXXX XXXX	1111	1100 – 1110 1111	0000 – 1111 0000 – 1101	A_1
1111 1111 111X XXXX	1111	1111	1110 – 1111	0000 - 1111

设地址长度为n，上一层留出m种状态，下一层可扩展出m×2^n种状态

1. ISA指令集体系结构 完整定义了软件与硬件的接口，没有规定的是：执行每条指令所包含的控制信号（由控制单元根据ISA生成）、CPU的时钟周期、加法器的进位方式（硬件电路）
2. 程序控制类指令：改变程序执行的顺序，包括无条件/有条件转移、子程序调用、返回指令、循环指令、陷阱等

4.2 指令的寻址方式

1. 指令寻址：

   ①顺序寻址：PC+1（1指令字长，实际和编址有关，如按字节编址，指令字长为16位，则是PC+2）

   ②跳跃寻址：绝对转移（直接给地址）、相对转移（PC偏移量）

2. 数据寻址：形式地址A，有效地址EA，都有取指令访存1次

   ①隐含寻址：ACC

   ②立即寻址：A就是操作数，访存1次

   ③直接寻址：直接存地址，EA=A，访存2次

   ④间接寻址：存的是地址的地址，EA=(A)，访存3次

   ⑤寄存器寻址：存的寄存器编号，EA=Ri，访存1次　执行阶段不用访存

   ⑥寄存器间接寻址：寄存器里存地址，EA=(Ri)，访存2次

   偏移寻址：

   ⑦**相对寻址**：相对PC（下一条指令）偏移，EA=(PC)+A，程序浮动（一段代码在程序内部的浮动），访存2次

   A偏移，可正可负，补码表示

   代码在程序内的浮动指的就是目标指令和当前指令地址的差异

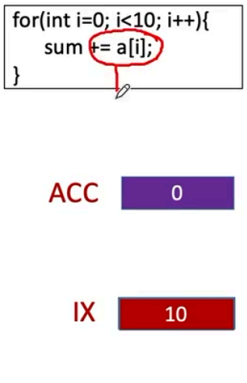
   ⑧**基址寻址**：相对基址寄存器BR面向OS，EA=(BR)+A，浮动程序（程序在内存的浮动），程序执行期间不可变，访存2次

   (BR)中是基址，A偏移

   程序在内存的浮动是os不一定每次都把程序加载到同一个物理地址

   ⑨**变址寻址**：相对变址寄存器IX面向用户，EA=(IX)+A，循环程序处理数组问题，**程序执行期间可变**，访存2次

(IX)是偏移量，A是基地址



	主存地址	指令		注释
		操作码	地址码	
立即寻址	0	取数到ACC	#0	立即数 0 → ACC
	1	取数到IX	#0	立即数 0 → IX
变址寻址	2	ACC加法	7（数组始址）	(ACC)+(7+(IX))→ ACC
立即寻址	3	IX加法	#1	(IX) + 1→ IX
	4	IX比较	#10	比较10-(IX)
直接寻址	5	条件跳转	2	若结果>0 则PC跳转到2
	6	从ACC存数	17	(ACC)→ sum变量
	7	*随便什么值*		a[0]
	8	*随便什么值*		a[1]
	9	*随便什么值*		a[2]

⑩堆栈寻址：软硬堆栈，堆栈指针SP（特定寄存器），无操作数指令，EA=(SP)+/-1，硬堆栈访存1次，软堆栈访存4次

3. 综合：若计算机16个通用寄存器，32位定长指令字，操作码8位，某个指令两个操作数分别寄存器直接寻址、基址寻址（基址寄存器可用任何一个通用），偏移量补码表示，则偏移量取值范围？

   8位op，则两个地址一共位，寄存器直接寻址用掉4位指出哪个寄存器，剩下20位都是基址寻址的，基址寻址的寄存器也占4位指出是哪个，剩下16位偏移 → -32768 ~ 32767

4.3 程序的机器级代码表示

2.【2019 统考真题】已知 $f(n)=n!=n\times(n-1)\times(n-2)\times\cdots\times2\times1$，计算 $f(n)$的 C 语言函数 f1 的源程序（阴影部分）及其在 32 位计算机 M 上的部分机器级代码如下：

```
int   f1(int n){
  1     00401000    55              push  ebp
        ...         ...             ...
        if(n>1)
  11    00401018    83 7D 08 01     cmp  dword ptr [ebp+8],1
  12    0040101C    7E 17           jle  f1+35h (00401035)
        return n*f1(n-1);
  13    0040101E    8B 45 08        mov  eax, dword ptr [ebp+8]
  14    00401021    83 E8 01        sub  eax, 1
  15    00401024    50              push  eax
  16    00401025    E8 D6 FF FF FF  call  f1 ( 00401000)
        ...         ...             ...
  19    00401030    0F AF C1        imul  eax, ecx
  20    00401033    EB 05           jmp  f1+3Ah (0040103a)
        else return 1;
  21    00401035    B8 01 00 00 00  mov  eax,1
}
        ...         ...             ...
  26    00401040    3B EC           cmp  ebp, esp
        ...         ...             ...
  30    0040104A    C3              ret
```

其中，机器级代码行包括行号、虚拟地址、机器指令和汇编指令，计算机 M 按字节编址，int 型数据占 32 位。请回答下列问题：

函数名作为标号，标记函数指令的起始地址，如f1实际上表示00401000

1. mov 目的d 源s，d不可以是常量，两个操作数不能都是内存
2. dword ptr --- 双字，32bit

   word ptr --- 单字，16bit

byte ptr --- 字节，8bit

3. 通用寄存器：累加器：AX,EAX，基地址寄存器（基址寻址）：BX,EBX，计数寄存器：CX,ECX，数据寄存器：DX,EDX　16bit,32bit

   变址寄存器：ESI,EDI（线性表，字符串处理）（变址寻址）

   堆栈基指针：EBP，堆栈顶指针ESP（函数调用）　32b

4. 任意寄存器，内存地址，常数，加数表位数

5. 数据传送指令：mov，push，pop

   算术运算指令：add/sub，inc/dec自增/减，imul/mul乘，idiv/div除（被除数隐含）

   逻辑运算指令：and/or/xor，not翻转，neg取负，shl/shr逻辑移位

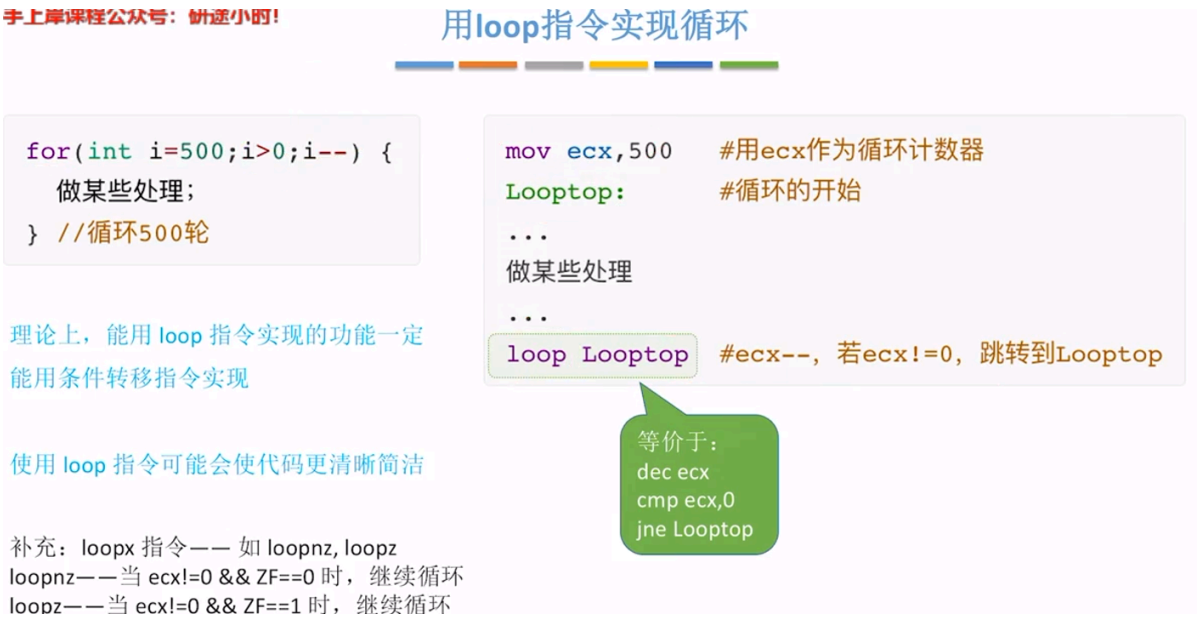
   控制流指令：jmp，j条件(je相等，jz零，jne不等，jg大于，jge大于等于，jl小于，jle小于等于)，cmp（减法）/test，call/test

   用于实现分支结构、循环结构的指令：cmp、test、jmp、jxxx

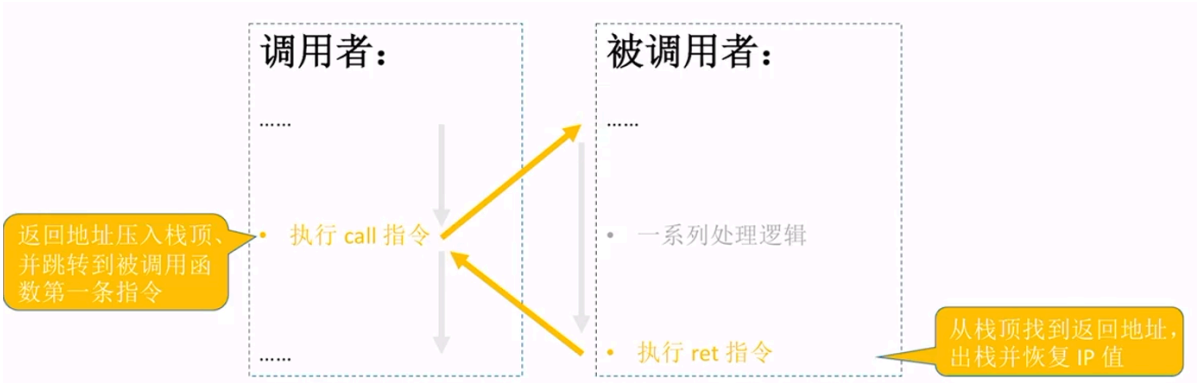
   用于实现函数调用的指令：push、pop、call、ret

   用于实现数据转移的指令：mov

6. 循环



7. 函数调用：



call 指令的作用：
①将IP旧值压栈保存（保存在函数的栈帧顶部）
②设置IP新值，无条件转移至被调用函数的第一条指令

ret 指令的作用：
从函数的栈帧顶部找到IP旧值，将其出栈并恢复IP寄存器

选择题

后续课程完整更新公众号：研途小时！

一手上岸课程公众号：研途小时！

访问栈帧数据：push、pop 指令

push、pop 指令实现入栈、出栈操作，x86 默认以4字节为单位。指令格式如下：

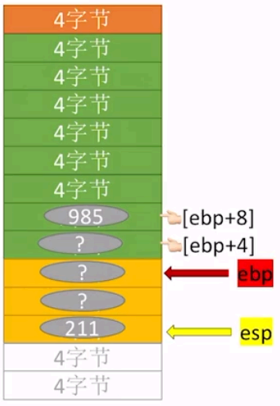
Push 🐶　　// 先让esp减4，再将 🐶 压入
Pop 🐴　　//栈顶元素出栈写入 🐴，再让 esp加4

注1：🐶可以是立即数、寄存器、主存地址
注2：🐴可以是寄存器、主存地址

例：

```
push eax          #将寄存器eax的值压栈
push 985          #将立即数985压栈
push [ebp+8]      #将主存地址[ebp+8]里的数据压栈

pop eax           #栈顶元素出栈，写入寄存器eax
pop [ebp+8]       #栈顶元素出栈，写入主存地址[ebp+8]
```



ebp：指向当前栈帧的“底部”

esp：指向当前栈帧的“顶部”

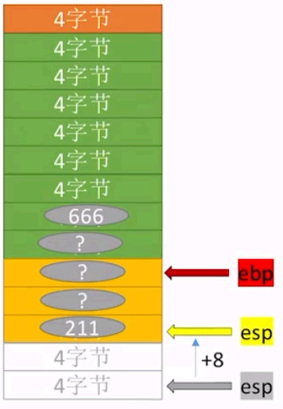
后续课程完整更新公众号：研途小时！

一手上岸课程公众号：研途小时！

访问栈帧数据：mov 指令

例：

```
sub esp, 12          #栈顶指针-12
mov [esp+8], eax     #将eax的值复制到主存[esp+8]
mov [esp+4], 958     #将985复制到主存[esp+4]
mov eax, [ebp+8]     #将主存[ebp+8]的值复制到eax
mov [esp],eax        #将eax的值复制到主存[esp]
add esp, 8           #栈顶指针+8
```



- 可以用 mov 指令，结合 esp、ebp 指针访问栈帧数据
- 可以用减法/加法指令，即 sub/add 修改栈顶指针 esp 的值

ebp：指向当前栈帧的“底部”

esp：指向当前栈帧的“顶部”

后续课程完整更新公众号：研途小时！

手上岸课程公众号：研途小时！

总结：函数调用的机器级表示

除了main函数，其他所有函数的汇编代码结构都一样！

调用者：

……

- 执行 call 指令

返回地址压入栈顶、并跳转到被调用函数第一条指令

……

被调用者：

- 保存上一层函数栈帧，设置当前函数栈帧

push ebp
mov ebp,esp
或：enter指令

- 一系列处理逻辑
- 恢复上一层函数的栈帧

mov esp, ebp
pop ebp
或：leave指令

- 执行 ret 指令

从栈顶找到返回地址，出栈并恢复 IP 值

手上岸课程公众号：研途小时！

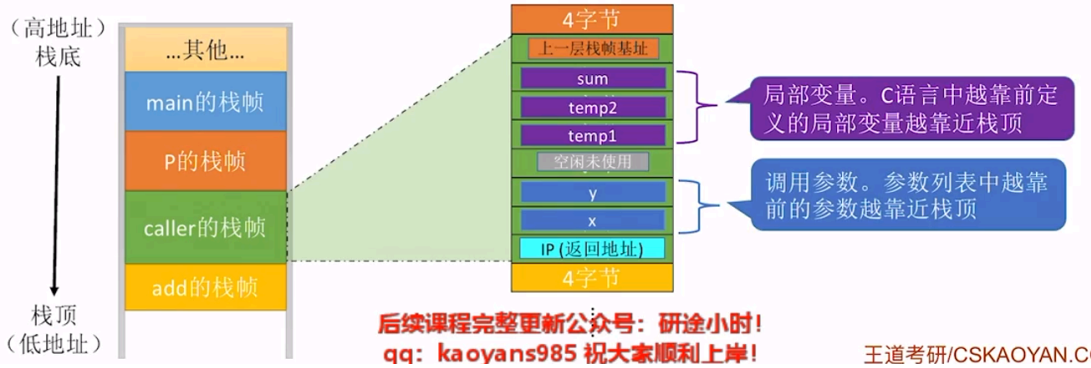
GNU/Linux家的好东西，开源、自由、非常流行

一个栈帧内可能包含哪些内容？

- gcc 编译器将每个栈帧大小设置为 16B 的整数倍（当前函数的栈帧除外），因此栈帧内可能出现空闲未使用的区域。
- 通常将局部变量集中存储在栈帧底部区域
- 通常将调用参数集中存储在栈帧顶部区域
- 栈帧最底部一定是上一层栈帧基址（ebp旧值）
- 栈帧最顶部一定是返回地址（当前函数的栈帧除外）

```
int caller(){
  int temp1=125;
  int temp2=80;
  int sum=add(temp1,temp2);
  return sum;
}

int add(int x, int y){
  return x+y;
}
```



后续课程完整更新公众号：研途小时！
qq：kaoyans985 祝大家顺利上岸！
王道考研/CSKAOYAN.COM

手上岸课程公众号：研途小时！

总结：函数调用的机器级表示

调用者：

……

保存必要的寄存器

如 eax、edx、ecx

- 将调用参数写入当前栈帧的顶部区域

可用 push 或 mov 指令实现

- 执行 call 指令

返回地址压入栈顶、并跳转到被调用函数第一条指令

- 使用返回值

通过eax寄存器

恢复必要的寄存器

……

被调用者：

- 保存上一层函数栈帧，设置当前函数栈帧

push ebp
mov ebp,esp
或：enter指令

- 初始化局部变量

[ebp-4]、[ebp-8]...

- 一系列处理逻辑
- 向上层函数传递返回值

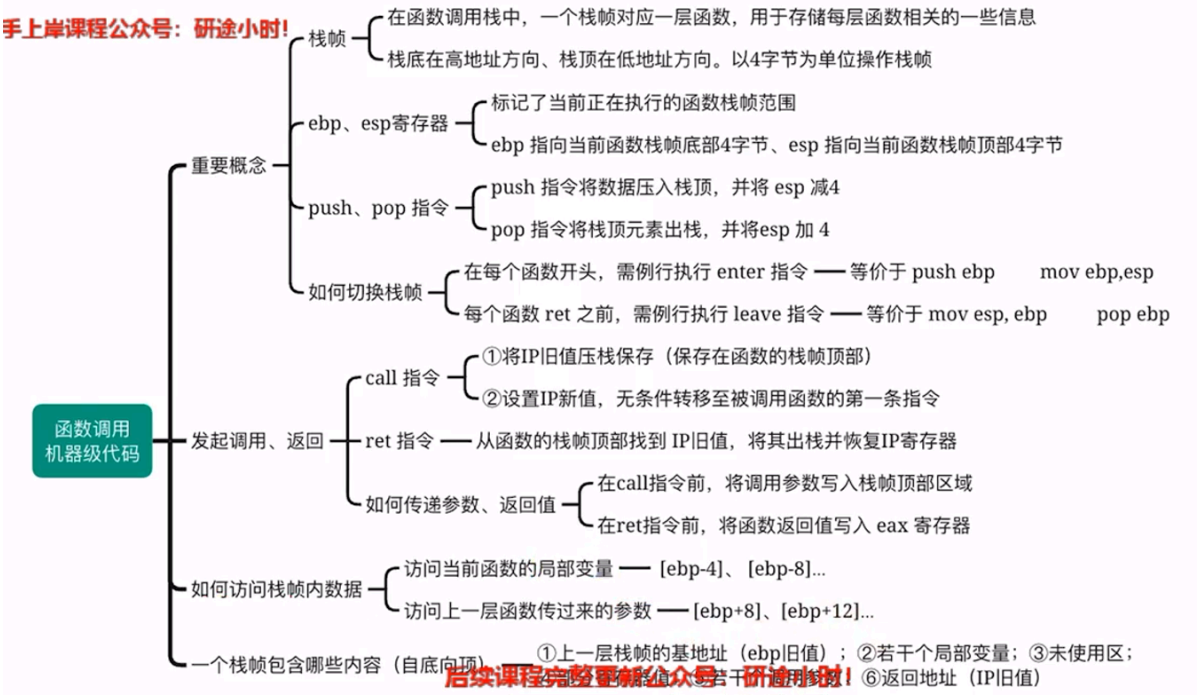
通过eax寄存器

- 恢复上一层函数的栈帧

mov esp, ebp
pop ebp
或：leave指令

- 执行 ret 指令

从栈顶找到返回地址，出栈并恢复 IP 值



4.4 CISC和RISC的基本概念

对比	复杂指令系统计算机CISC	精简指令系统计算机RISC
指令字长	不固定	固定
可访存指令	不限制	只有LOAD/STORE指令
通用寄存器数量	较少	多
控制方式	绝大多数为微程序控制	绝大多数为组合逻辑控制（硬布线）
指令系统兼容	大多可以	大多不行
指令流水线	可以通过一定方式实现	必须实现
目标代码	难以用优化变异生成高效的目标代码程序	采用优化的编译程序，生成代码更为高效
各指令执行时间	相差较大	绝大多数一个周期内

1. RISC指令功能简单，通过简单指令的组合来实现复杂指令的功能，而不是以增加每条指令功能的方法。是CISC有增强指令的功能

5 中央处理器

5.1 CPU的功能和基本结构

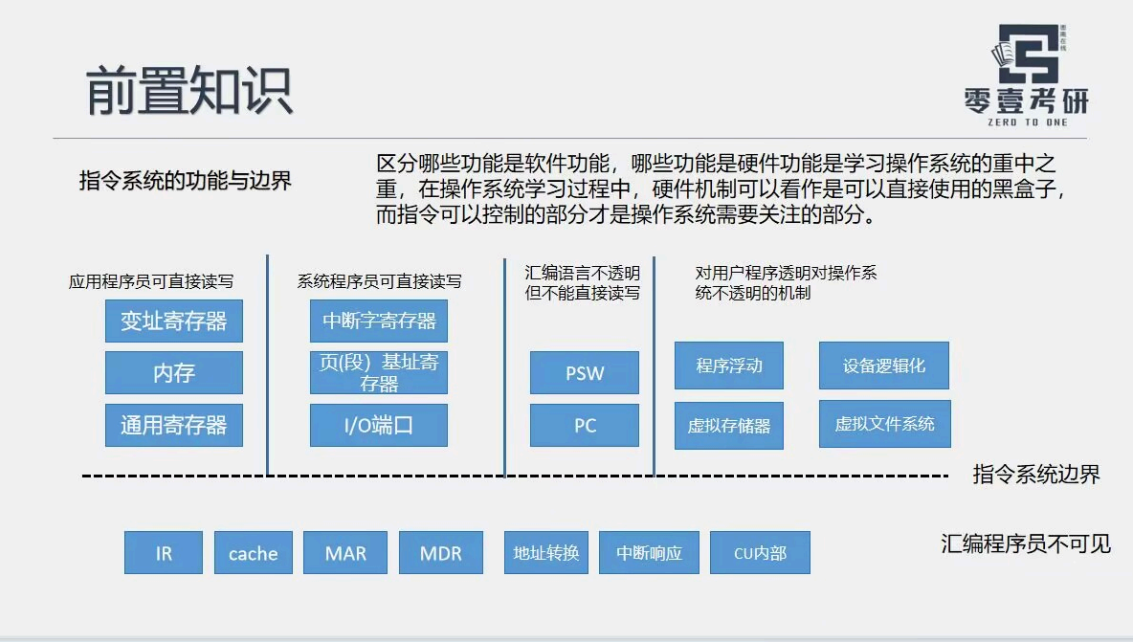
1. 中央处理器CPU = 运算器 + 控制器，实现指令控制、操作控制、时间控制、数据加工、中断处理

2. 用户可见寄存器：**GPRs通用寄存器组，PSW程序状态字寄存器/标志寄存器，ACC累加寄存器，PC程序计数器**，SR移位寄存器

   用户不可见寄存器：MAR，MDR，IR指令寄存器，暂存寄存器

   汇编程序员可见：基址寄存器，PC，状态寄存器，通用寄存器

   汇编程序员不可见：IR，MAR，MDR



3. 运算器中：寄存器（GPRs,ACC,SR,暂存,PSW），ALU，计数器CT

   控制器中：寄存器（PC,IR,MAR,MDR），指令译码器ID（译的是操作码），时序电路，微操作信号发生器

4. 标志寄存器PSW是专用寄存器，不需要编号，也不能在指令中指定编号来访问，用户不能直接修改它的值

5. 指令总是根据PC从主存储器中读出，如遇到转移指令，只需用跳转地址覆盖原PC的内容即可

5.2 指令执行过程

1. 1指令周期（取出并执行一条指令）= m机器周期（即CPU周期）= mn时钟周期（即CPU时钟周期，节拍，基本单位）

   1指令周期 = 取指周期FE（根据PC取代码放IR）+ 间址周期IND（取操作数有效地址）+ 执行周期EX + 中断周期INT（处理中断请求）

   （这四个周期分别都不一定是一个机器周期，如指令字长是存储字长2倍，则取指周期等于机器周期2倍）

2. 单周期处理器（CPI(1条指令所需时钟周期)=1，指令间串行，指令周期固定=最长的），多周期处理器（CPI>1，分步骤，仍串行执行），流水线处理器（理想CPI=1，尽量多条指令同时运行）

3. 计算机中指令和数据均以二进制形式存放在存储器中，CPU区分它们的依据是：指令周期的不同阶段，取指阶段是指令，执行阶段是数据，由控制器来区分。

5.3 数据通路的功能和基本结构

1. **CPU = 数据通路 + 控制部件**

数据通路：数据在指令执行过程中所经过的路径，包括路径上的部件，包括：

组合逻辑元件（**操作元件**）：输出仅取决于输入，不含存储信号的记忆单元，无反馈通路，信号单向传输，**ACC,ALU,MUX,译码器,三态门**

时序逻辑元件（**状态元件**）：输出和以前的输入有关，包含存储信号的记忆单元，**PC**，**寄存器**组，状态/移位/暂存/锁存寄存器

总结：数据通路包括：ALU、通用寄存器、状态寄存器、Cache、**MMU**、浮点运算逻辑、**异常和中断处理逻辑**.....

2. 数据通路基本结构：CPU内部！ CPU内部单总线方式（单周期处理器不能采用），CPU内部多总线方式，专用数据通路方式

内部总线：同一部件内 vs 系统总线：同一台计算机系统的各部件

5.4 控制器的功能和工作原理

1. 硬布线控制器/组合逻辑控制器：

控制单元CU的输入信号来源：1.IR中操作码经译码器译码产生的指令信息，2.时序系统产生的机器周期信号和节拍信号，3.来自执行单元的反馈信息即标志（PSW,ACC,I/O设备,主存...）

功能由逻辑门组合实现，速度取决于电路延迟（快），RISC采用，增加修改麻烦

2. 微程序控制器：

①微命令和微操作是一一对应的。1微指令=n微命令，包括操作控制字段、顺序控制字段

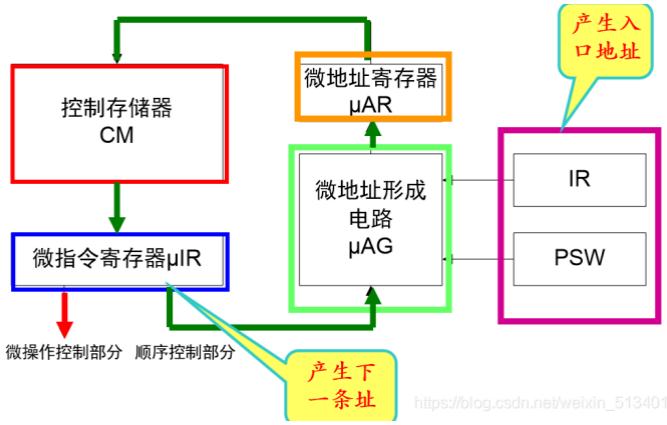
②控制存储器CM 用于存放微程序，用**ROM**实现，---微地址

③程序是指令的有序集合，微程序是微指令的有序集合，一条指令的功能由一段微程序来实现，微程序实时解释机器指令

④微指令编码/控制方式：直接编码方式（一位对应一个微命令），字段直接编码方式（互斥同一字段，相容不同字段，每段留一个状态），字段间接编码方式/隐式编码

⑤微指令地址形成方式：**断定方式（后继地址字段给出）**，操作码，μPC+1，根据各种标志决定分支转移，硬件直接产生微程序入口地址

⑥微指令的格式：水平型微指令（④都是，定义并执行多个并行操作的微命令，字），垂直型微指令（加了μOP微操作码+Rd+Rs，通常一对一微命令），混合型微指令



3. 微程序控制器的时序系统比较简单

4. 在微程序控制器中，形成微程序入口地址的是机器指令的操作码字段

5. 取指令操作是控制器固有的功能，不需要在操作码控制下进行

5.5 异常和中断机制（7.3）

1. **异常**：内中断，CPU内部的意外事件，与正在执行的**指令相关**

   （1）故障：指令启动后结束前，返回时到当前发生异常的指令。非法操作码(终止)、除0(终止)、缺页缺段(回到原指令继续执行)、运算结果异常(如溢出)

   （2）自陷：预先安排，返回时到下一条指令/转移目标指令。程序的断点调试与单步跟踪、系统调用指令、条件自陷指令 → 转内核程序

   （3）终止：随机发生。控制器出错、存储器校验错、总线错误 → 终止+调出异常服务程序来重启系统

2. **中断**：外中断，CPU外部设备发出的中断请求，与正在执行的**指令无关**。返回时到已经执行完成的指令的下一条指令

   I/O设备发出的I/O中断（键盘输入、打印机缺纸）+ 特殊事件（定时器计数时间到，按Esc键）

   （1）可屏蔽中断：INTR请求线

   （2）不可屏蔽中断：NMI请求线。电源掉电

3. ① 硬故障中断：存储器校验错，总线错误：终止+外中断

   ② 程序性异常（软件中断）：故障+陷入

4. 所有异常和中断事件都是由**硬件检测**发现的，整个中断处理过程是软件硬件协同实现的

5. 异常和中断响应： 详见7.3

   ①关中断：设置中断允许触发器IF=0

   ②保存断点和程序状态：保存地址、PSW

   ③识别异常和中断并转到相应的处理程序：

   软件识别（异常大多、中断）：CPU中设置一个**异常状态寄存器**，操作系统用统一的查询程序来查询并转到内核运行（系统调用就是，OS第一章）

   硬件识别（中断）：即**向量中断**。根据中断类型号找到对应中断向量（程序首址）

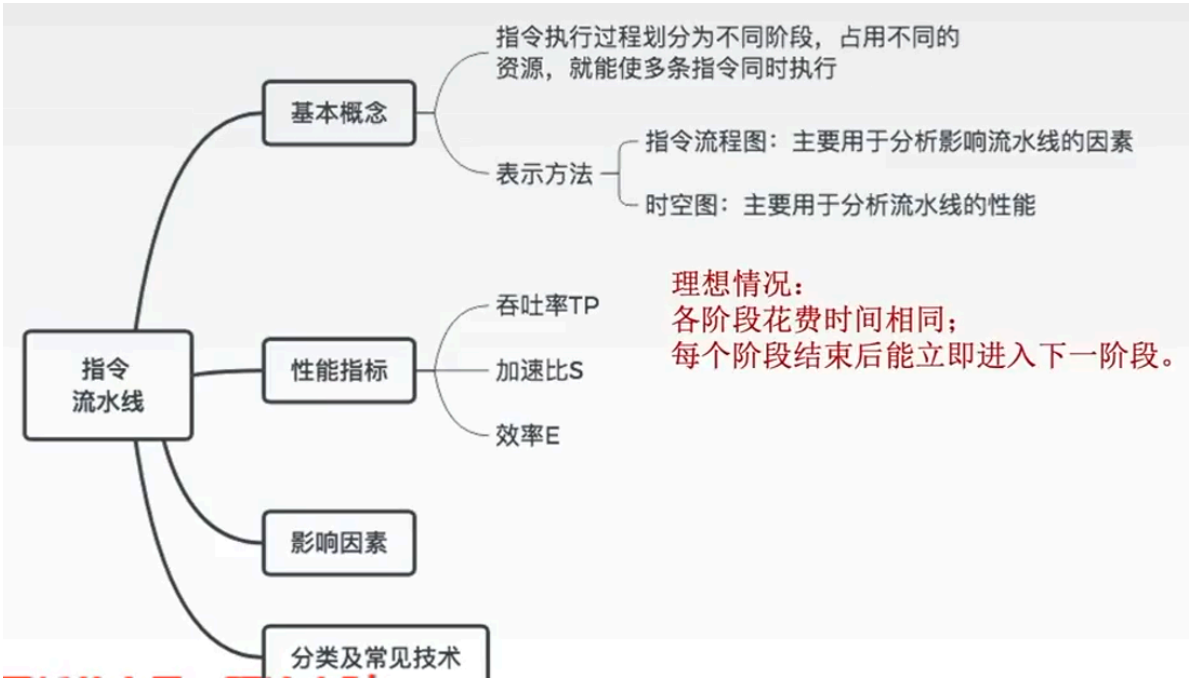
6. TLB、Cache缺失的处理是由CPU硬件实现的，不是异常也不是中断，更无须调出异常处理程序进行处理

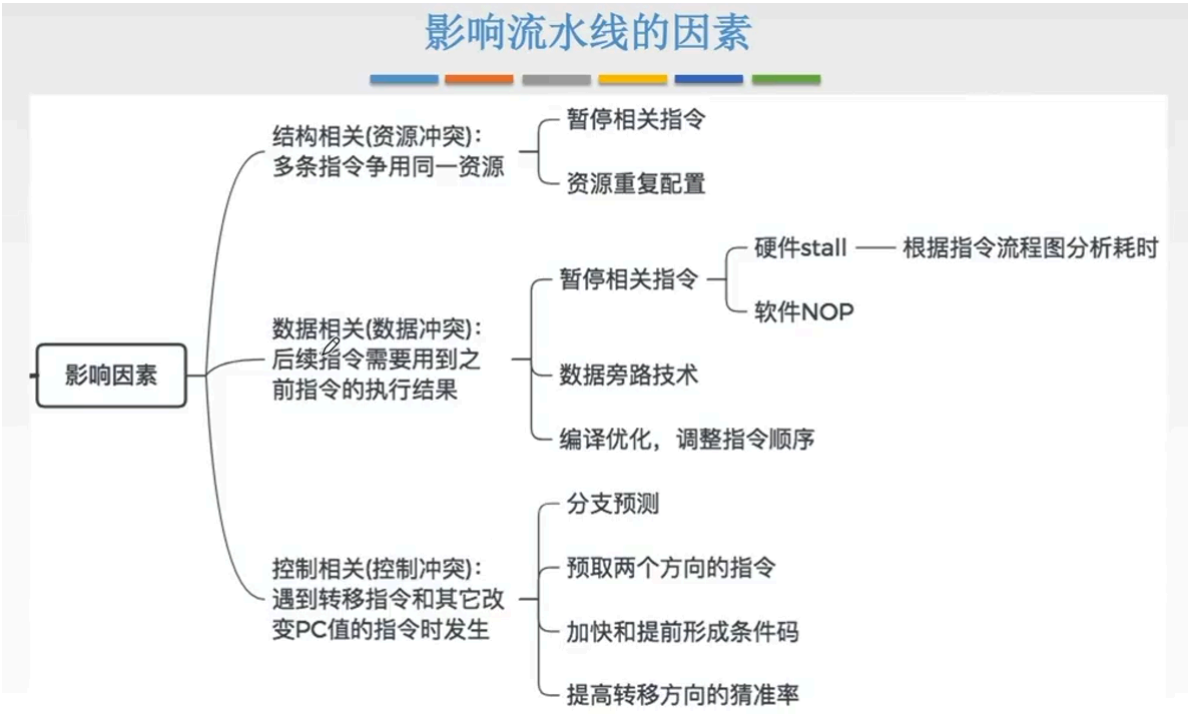
7. 进程上下文切换由操作系统的内核程序实现，而异常/中断的响应（即关中断、保存断点和程序状态）由硬件实现，响应后执行的是异常/中断服务程序，服务程序是操作系统的内核程序

8. 中断一定是在一条指令执行结束后开始查询有无中断请求，有的话立即响应，所以一定是在指令执行完才进行异常处理

   异常发生在指令执行过程中，所以不能等到指令执行完才进行异常处理

5.6 指令流水线

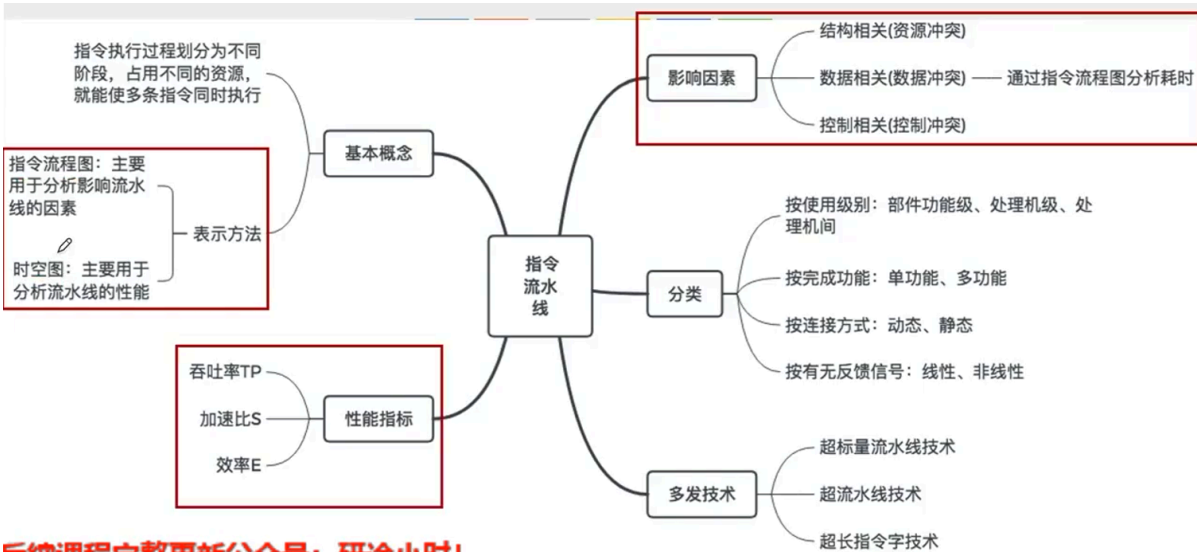




[袁春风]解决结构冒险/资源冲突：

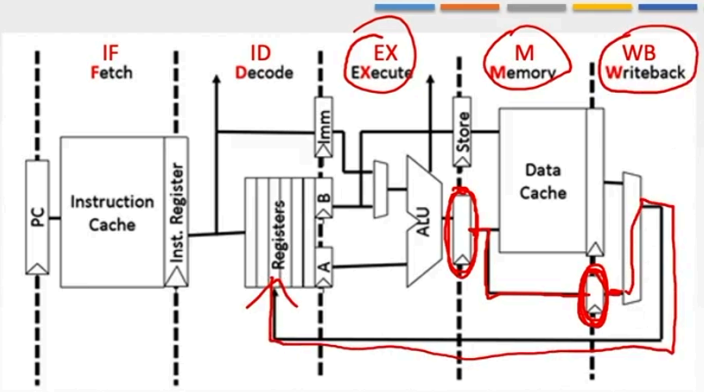
①一个部件每条指令只能使用一次，且只能在特定阶段使用，可以避免一部分结构冒险（/使每个指令在相同流水段中使用相同的部件 √）

②通过设置多个独立的部件来避免硬件资源冲突，例如可以将寄存器的读口和写口分开，将指令Cache和数据Cache分离等



RISC下的指令操作：

运算类指令的执行过程



注：
R_s 指源操作数（source）
R_d 指目的操作数（destination）

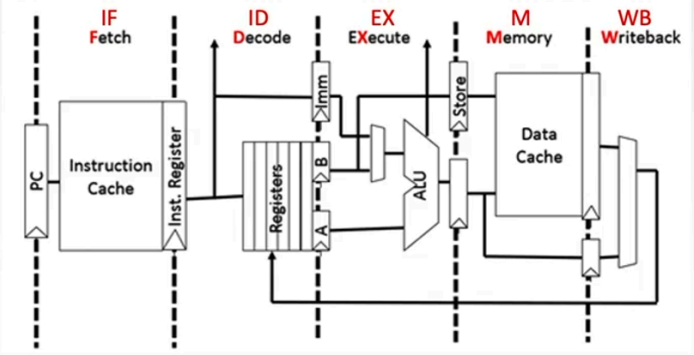
运算类指令
IF：根据PC从指令Cache取指令至IF段的锁存器
ID：取出操作数至ID段锁存器
EX：运算，将结果存入EX段锁存器
M：空段
WB：将运算结果写回指定寄存器

①IF取指 → ②ID译码&取数 → ③EX 执行→ ~~④M访存~~ → ⑤WB写回寄存器

运算类指令举例	指令的汇编格式	功能
加法指令（两个寄存器相加）：	ADD Rs,Rd	(Rs)+(Rd)→Rd
加法指令（寄存器与立即数相加）：	ADD #996,Rd	996+(Rd)→Rd
算数左移指令：	SHL Rd	(Rd)<<<2→Rd

课程完整更新公众号：研途小时！

LOAD指令的执行过程



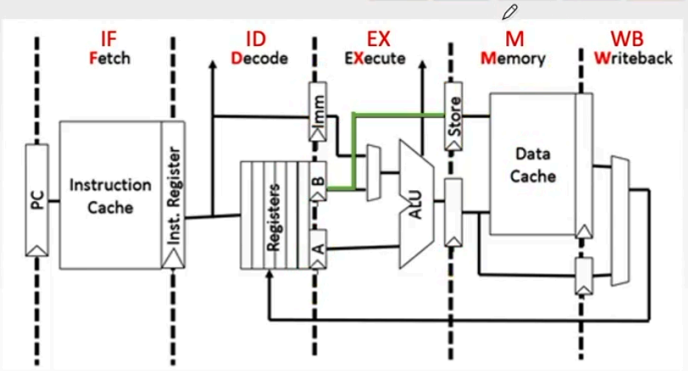
LOAD指令
IF：根据PC从指令Cache取指令至IF段的锁存器
ID：将基址寄存器的值放到锁存器A，将偏移量的值放到Imm
EX：运算，得到有效地址
M：从数据Cache中取数并放入锁存器
WB：将取出的数写回寄存器

①IF取指 → ②ID译码&取数 → ③EX 执行→ ④M访存 → ⑤WB写回寄存器

指令的汇编格式	功能
LOAD Rd,996(Rs)	(996+(Rs))→Rd

通常，RISC处理器只有“取数LOAD”和“存数STORE”指令才能访问主存

STORE指令的执行过程



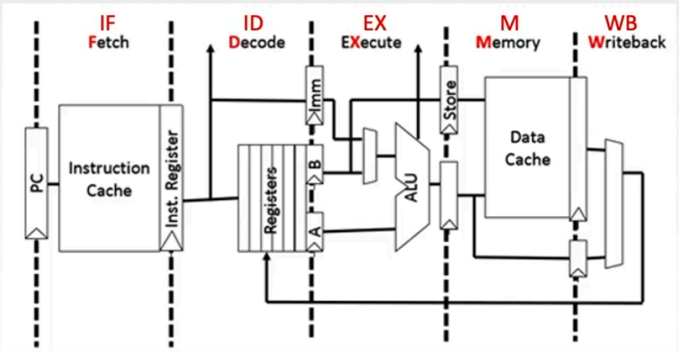
STORE指令
IF：根据PC从指令Cache取指令至IF段的锁存器
ID：将基址寄存器的值放到锁存器A，将偏移量的值放到Imm。将要存的数放到B
EX：运算，得到有效地址。并将锁存器B的内容放到锁存器 Store。
M：写入数据Cache
WB：空段

①IF取指 → ②ID译码&取数 → ③EX 执行→ ④M访存 → ~~⑤WB写回寄存器~~

指令的汇编格式	功能
STORE Rs,996(Rd)	Rs→(996+(Rd))

条件转移指令的执行过程

转移类指令常采用相对寻址



条件转移指令
IF：根据PC从指令Cache取指令至IF段的锁存器
ID：进行比较的两个数放入锁存器A、B；偏移量放入 Imm
EX：运算，比较两个数
M：将目标PC值写回PC（左图没画全）
WB：空段

很多教材把写回PC的功能段称为“WrPC段”，其耗时比M段更短，可安排在M段时间内完成

①IF取指 → ②ID译码&取数 → ③EX 执行→ ④M访存 → ~~⑤WB写回寄存器~~

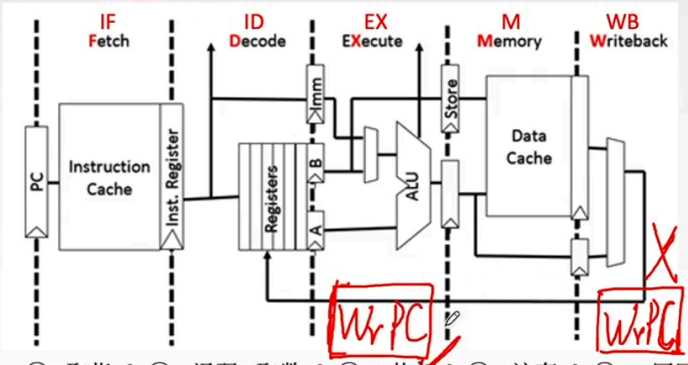
注：通常在IF段结束止之后PC就会自动 + “1”

指令的汇编格式	功能
beq Rs, Rt, #偏移量	若(Rs)==(Rt),则(PC)+指令字长+(偏移量×指令字长)→PC；否则(PC)+指令字长→PC
~~bne Rs,Rt,#偏移量~~	若(Rs)!=(Rt),则(PC)+指令字长+(偏移量×指令字长)→PC；否则(PC)+指令字长→PC

课程完整更新公众号：研途小助

无条件转移指令的执行过程

转移类指令常采用相对寻址



无条件转移指令
IF：根据PC从指令Cache取指令至IF段的锁存器
ID：偏移量放入 Imm
EX：将目标PC值写回PC（左图没画全）
M：空段
WB：空段

①IF取指 → ②ID译码&取数 → ③EX 执行→ ~~④M访存~~ → ~~⑤WB写回寄存器~~

指令的汇编格式	功能
jmp #偏移量	(PC)+指令字长+(偏移量×指令字长)→PC

“WrPC段”耗时比EX段更短，可安排在EX段时间内完成。WrPC段越早完成，就越能避免控制冲突。当然，也有的地方会在WB段时间内才修改PC的值

例题. 假设某指令流水线采用“按序发射，按序完成”方式，没有采用转发技术处理数据相关，并且同一寄存器的读和写操作不能在同一个时钟周期内进行。若高级语言程序中某赋值语句为x=a+b，x、a和b均为int型变量，它们的存储单元地址分别表示为[x]、[a]和[b]。该语句对应的指令序列及其在指令流中的执行过程如下图所示。

```
I1   LOAD   R1, [a]    M[a] —> R1
I2   LOAD   R2, [b]    M[b] —> R2
I3   ADD    R1, R2     ( R1 ) + ( R2 ) —> R2
I4   STORE  R2, [x]    ( R2 ) —> M[x]
```

则这4条指令执行过程中I3的ID段和I4的IF段被阻塞的原因各是什么？

	时间单元													
指令	1	2	3	4	5	6	7	8	9	10	11	12	13	14
I_1	IF	ID	EX	M	WB									
I_2		IF	ID	EX	M	WB								
I_3			IF				ID	EX	M	WB				
I_4							IF				ID	EX	M	WB

I3与I1和I2存在数据相关；

I4的IF段必须在I3进入ID段后才能开始，否则会覆盖IF段锁存器的内容

1. 单周期CPU：CPI=1。多周期CPU：CPI>1。基本流水线：CPI=1。超标量流水线：**CPI<1**。超流水线：**CPI=1**但主频更高，周期短了

   ① 超标量流水线技术：动态多发射技术，设置多个功能部件，指令并行，通过动态分支预测不按顺序执行（乱序执行），空分复用

   ② 超长指令字技术：静态多发射技术，多条并行操作指令合成一条超长指令字

   ③ 超流水线技术：更短的时钟周期，更多段，不能调整指令执行顺序，时分复用

2. load-use冒险可以只停一个周期？？？？？直接在EX段时从MEM/WB流水段寄存器中取数（前半个周期即上升沿load取出数到MEM/WB流水段寄存器，后半个周期即下降沿EX段直接从中取数重定位）

3. 在某个时钟周期内，不同的流水段受到不同指令的控制信号控制，执行不同指令的不同功能段，在指令译码阶段由控制器产生指令各流水段的所有控制信号，分别在随后的各个时钟周期内被使用，因此随后各流水段寄存器都要保存相应的控制信号，并通过流水段寄存器传递到下一个流水段。

4. 流水线数据通路中，时钟信号：作用在流水段寄存器、PC、存储器等等上。控制信号在译码后才产生，**IF和ID阶段是公共流水段，不需要控制信号**

撸起，可能导致数据冒险
绿色，可能导致控制冒险

如何分析数据冒险、控制冒险？

	IF（取指）	ID（译码/取数）	EX（执行/计算地址）	M（访存）	WB（写回）
Load指令	取指令	译码、取数（可能会读寄存器）	计算访存地址	从主存（Data Cache）读数据	将数据写回某个寄存器
Store指令	取指令	译码、取数（可能会读寄存器）	计算访存地址	往主存（Data Cache）写数据	空
转移类指令	取指令	译码、取数（可能会读寄存器）	计算转移地址	更新PC值	空
运算类指令	取指令	译码、取数（可能会读寄存器）	执行（算数运算、逻辑运算等）	空	将数据写回某个寄存器

注：熟悉四类指令在各个段分别做了什么

后续新课 添加微信：kaoyan929

数据冒险的分析和处理

考数据冒险？

1.分析是否存在数据冒险？

核心特点：两条指令，前“写”后“读”同一个寄存器。

分析方法：从第一条指令开始，结合注释，观察该指令“写”了某个寄存器，并观察后续相邻的3条指令是否“读”了同一个寄存器。若发现此类情况，则存在数据冒险。

2.常见的解决数据冒险的方法

a. 硬件阻塞：将“读寄存器指令”的ID段硬件阻塞若干个时钟，确保其ID段在“写寄存器指令”的WB段之后

b. 转发（旁路）技术

i. 转发（旁路）技术可以解决大部分数据冒险？

ii. 转发（旁路）技术不能解决由 Load 指令写寄存器引起的 Load-use 数据冒险？

数据旁路技术直接上一条EX计算完之后就转回到ALU寄存器，下一条指令在ID段取的数虽然是错误的，但是到EX段时能直接使用正确的，直接就不需要硬件阻塞了。

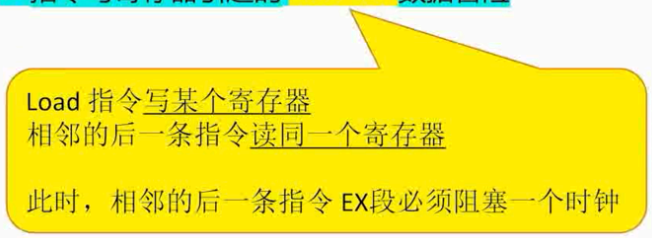
如果是Load-use，没办法完全解决，必须在上一条M访存取出之后再加上旁路，达到只停一个时钟周期的效果，或者直接程序编译优化调整指令顺序，如下：

真题实战：数据冒险——采用转发技术

注意：

i. 转发（旁路）技术可以解决大部分数据冒险

ii. 转发（旁路）技术不能解决由 Load 指令写寄存器引起的 Load-use 数据冒险



控制冒险注意阻塞是直接从IF段就阻塞3个到M即已经修改完后（数据冒险是ID段阻塞3个到WB即已经写回后，而采用数据旁路的load-use是EX段阻塞1个到M即已经读出后）

控制冒险的分析和处理

考控制冒险？

1.分析是否存在控制冒险？

只有转移类指令的执行才会引发控制冒险。因此只需观察指令序列中是否包含“转移类指令”即可。

2.常见的解决控制冒险的方法

硬件阻塞：将转移类指令后一条指令的IF段硬件阻塞3个时钟

5.7 多处理机的概念

1. 单指令流单数据流结构 SISD：传统串行计算机，各指令序列并发不并行，每条指令处理一两个数据，多模块交叉存储器+流水线
2. 单指令流多数据流结构 SIMD：数据级并行，一个指令控制部件CU，多个处理单元ALU，各自的局部存储器，一个主存储器。同时处理多个相同特征数据，指令序列并发不并行，如像素点的重复渲染，如for循环处理数组。
3. 多指令流单数据流结构 MISD：不存在
4. 多指令流多数据流结构 MIMD：线程级(及以上)并行，指令序列并行

   ①多计算机系统：消息传递MIMD，分布式，多处理器+多存储器，物理地址空间独立

   ②多处理器系统：共享存储多处理器SMP，共享存储MIMD，多处理器+一个主存储器，共享物理地址空间。通过LLC/主存来传消息
5. 向量处理器（←SIMD）：指令处理单位是向量（一维数组），数据并行计算，浮点数运算
6. 共享内存多处理器 SMP（**共享单一地址空间**，可通过访存指令访问共享的主存） = 多核处理器（所有核共享一个LLC 底层Cache）
7. 硬件多线程：指在一个核中处理多个线程，可用于单核处理器

硬件多线程			
	细粒度多线程	粗粒度多线程	同时多线程（SMT）
指令发射	各个时钟周期，轮流发射多个线程的指令	连续几个时钟周期，都发射同一线程的指令序列，流水线阻塞时，切换另一个线程	一个时钟周期内，同时发射多个线程的指令
线程切换频率	每个时钟周期切换一次线程	只有流水线阻塞时才切换一次线程	NULL
线程切换代价	低	高，需要重载流水线	NULL
并行性	指令级并行，线程间不并行	指令级并行，线程间不并行	指令级并行，线程级并行

8. 并行处理机、阵列处理机、向量处理机 ∈ SIMD

   超标量流水线处理机 ∈ SISD

超线程 ∈ 同时多线程，一个单处理器/单核中设置两套线程状态不见，共享高速缓存和功能部件

9. 多任务操作系统，即多道程序系统，可以运行在单核CPU上，宏观并行，微观串行
10. 时间并行：流水线技术

    空间并行：硬件资源的重复 → 导致SIMD,MIMD的产生

6 总线

6.1 总线概述

1. 总线特点：分时，共享

   总线设备：主设备，从设备

   总线特性：机械特性，电气特性，功能特性，时间特性

2. 总线分类：

   ①片内总线（芯片内部），系统总线（计算机系统各功能部件，数据总线、地址总线、控制总线），I/O总线（分离高速低速设备），通信总线（外部总线）

   I/O总线分为：

   数据线：数据缓冲寄存器和命令/状态寄存器的内容（**命令字**、状态字），还有中断类型号

   地址线：传送与CPU交换数据的端口地址

   控制线：给I/O端口发送读/写信号，**只是用来对端口进行读/写控制**

   ②串行总线（USB，长距离），并行总线（CPU和主存间，近距离）

   ③同步总线，异步总线

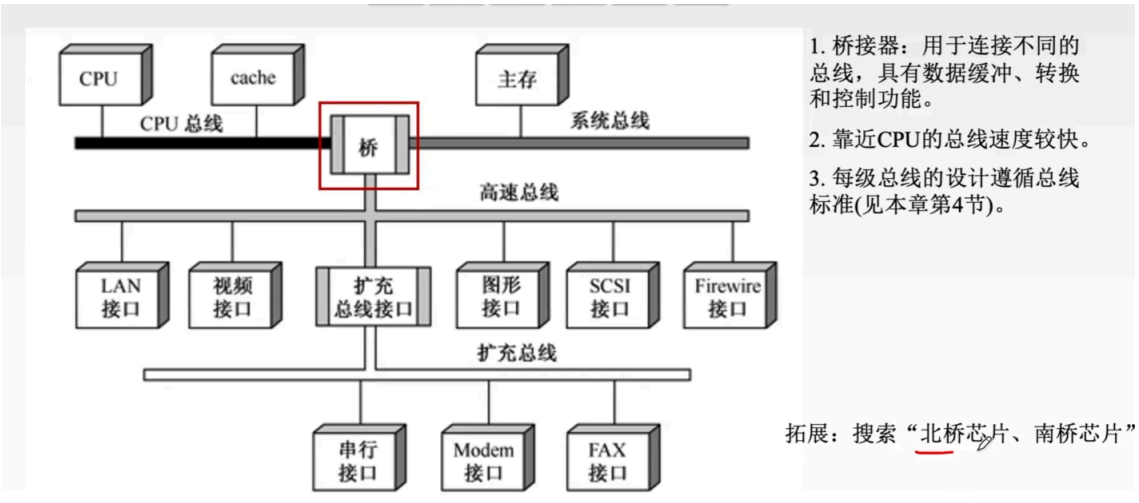
3. 系统总线结构：

   ①单总线结构：并非只有一根信号线

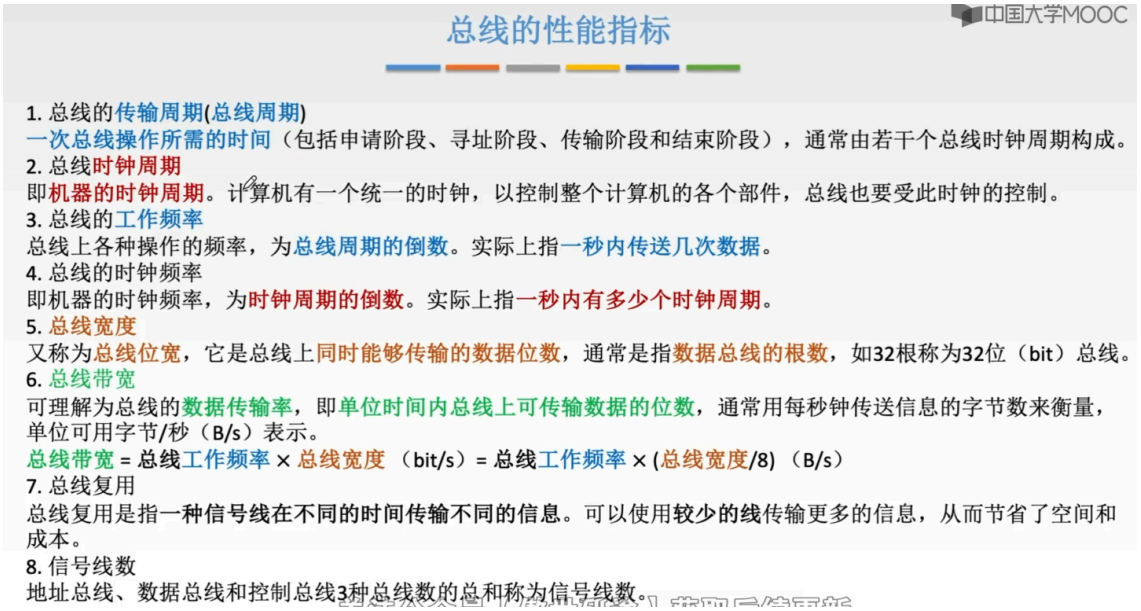
   ②双总线结构：一条主存总线（支持突发传送），一条I/O总线（通过通道连接主存总线）

   ③三总线结构：主存总线，I/O总线，DMA总线（内存和高速外设之间）

   ④四总线结构：



4. 总线性能指标：



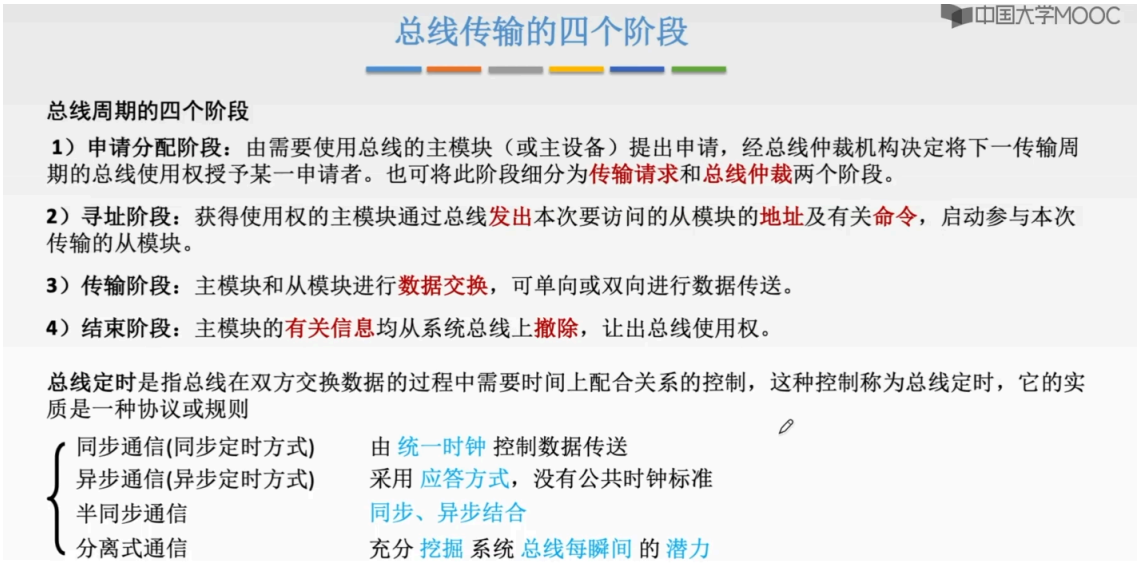
总线带宽：指最大数据传输率，即理想情况下（所有总线周期都在传数据时）的速率

vs（平均）数据传输率：需要考虑题中给出的每个总线事务的具体情况

5. 在一个总线周期内，总线上只有一个主设备控制总线，一对一或广播通信

6.2 总线事务和定时

1. 总线事务与总线定时：



①同步定时方式：统一的时钟信号，强制性同步，适合总线长度短，存取时间接近

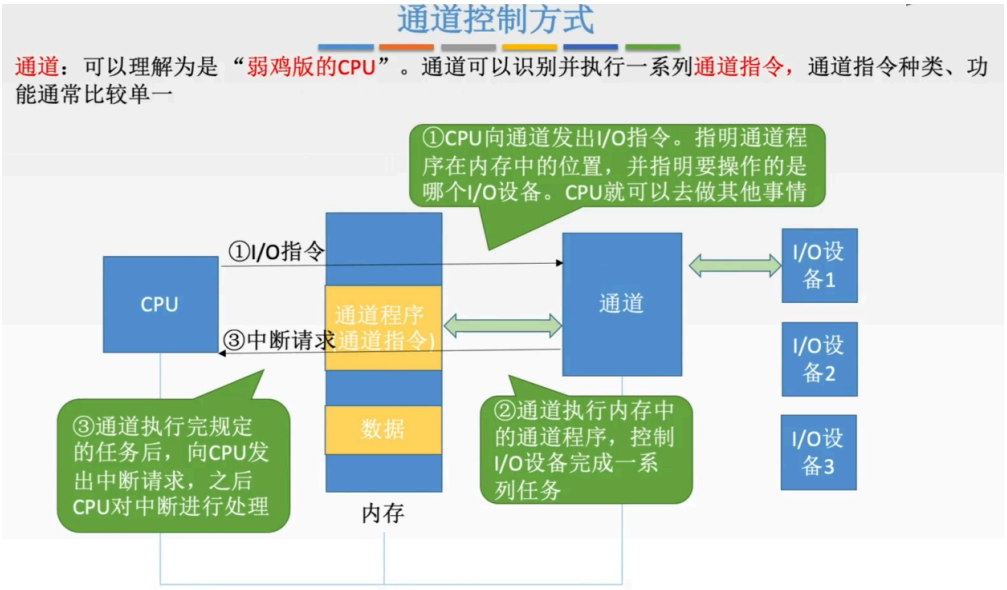
②异步定时方式：没有统一时钟，不互锁方式、半互锁方式、全互锁方式。全互锁才存在互锁关系。握手方式通信，一次握手完成一次通信，每次通信交换多位数据。

③半同步定时方式：加一条**Wait响应信号线**，在数据没准备好时候可以延长，每个时钟到来采样信号，握手信号的采样由同步时钟控制

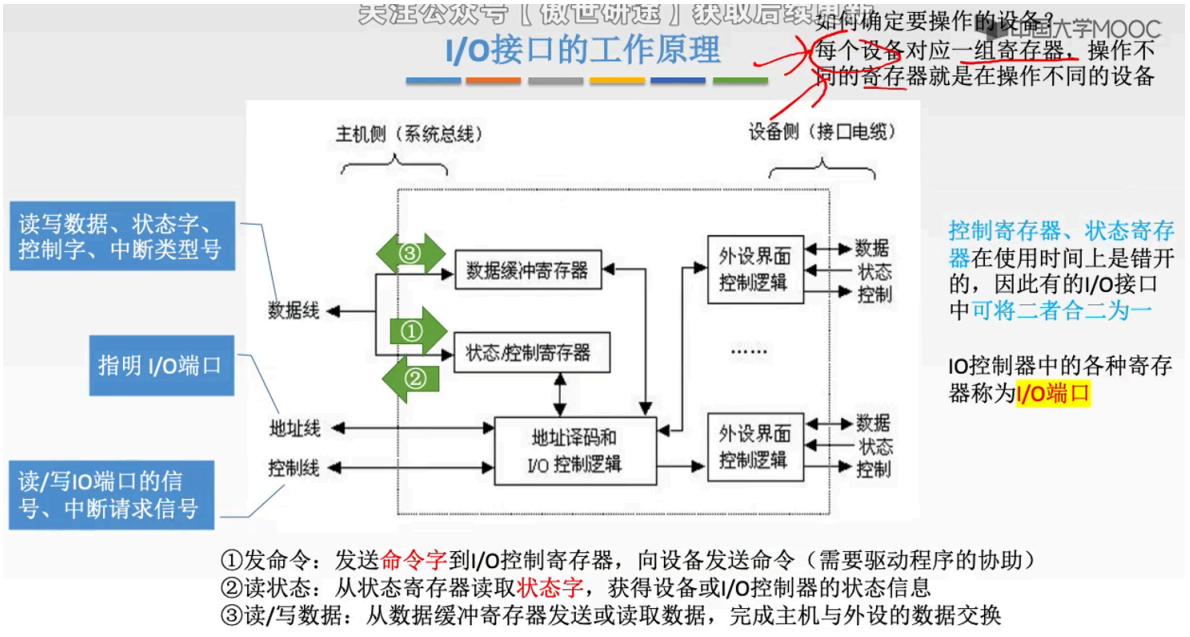
④分离式定时方式：总线事务分解为请求 和 应答子过程，每个设备都是主设备，请求完立即释放，对方准备好数据再申请，∈总线复用

7 输入输出系统

7.1 I/O系统基本概念（已删）



7.2 I/O接口



I/O端口：指I/O接口电路中可以被CPU直接访问的寄存器，数据端口、状态端口、控制端口

①独立编址：**I/O映射方式**，专门I/O指令（属于CPU系统指令），不占主存空间

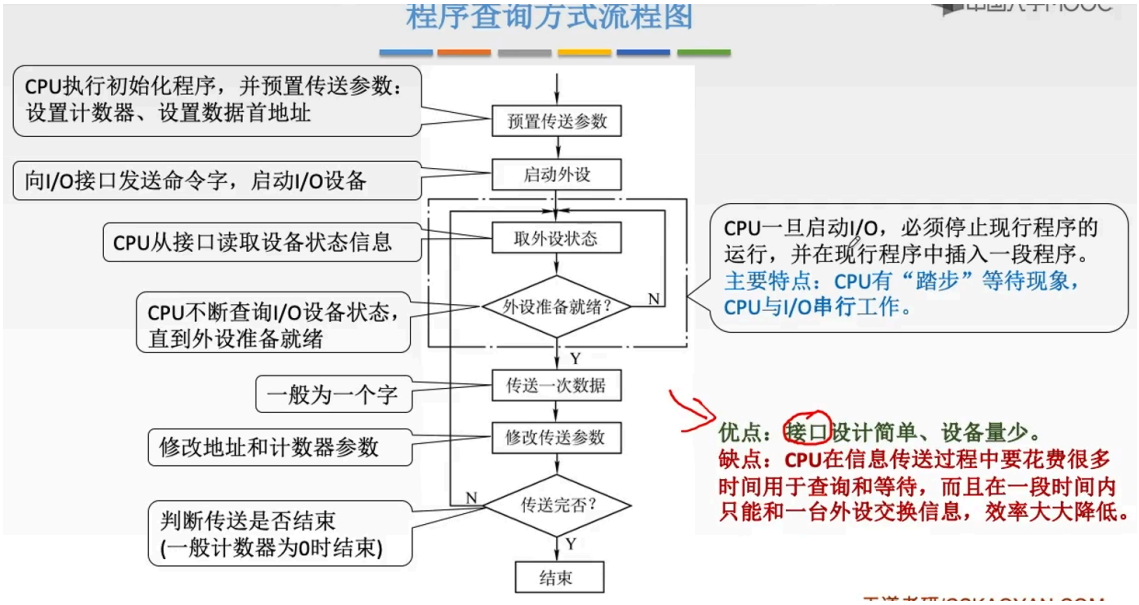
②统一编址：**存储器映射方式**，统一访存指令可以进行输入输出操作，RISC（CISC也可以），灵活方便，无需设置保护机制

磁盘驱动器不属于I/O接口

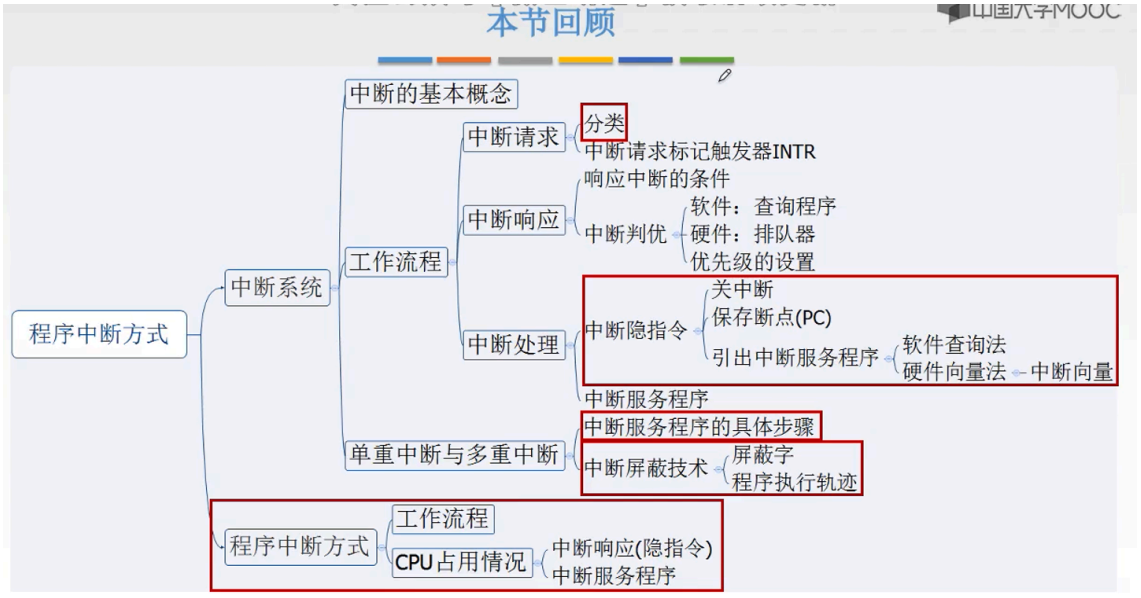
一个I/O端口对应一个地址；一个I/O接口可能对应多个地址

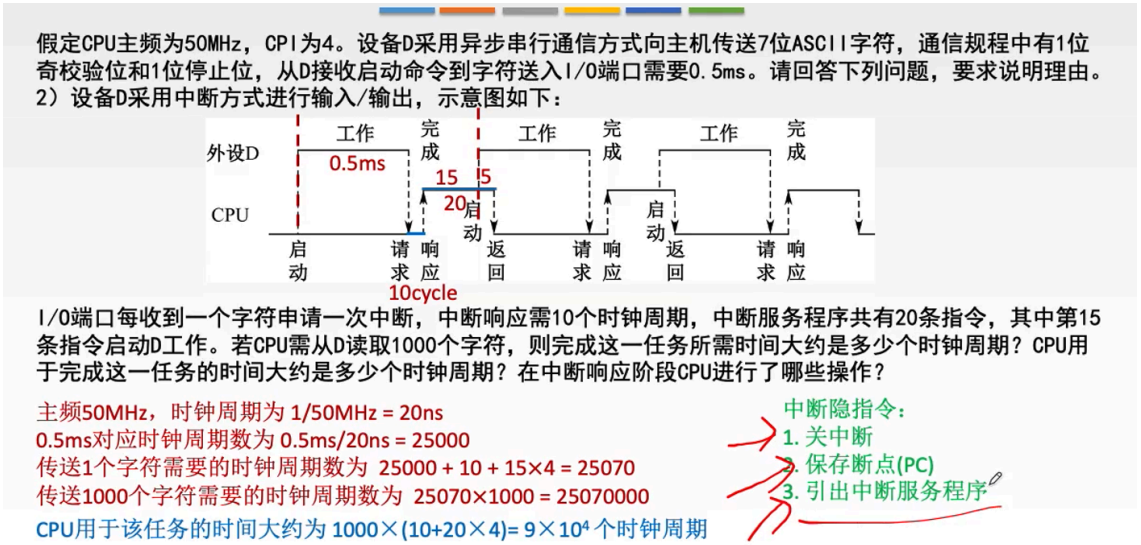
7.3 I/O方式

1. 程序查询方式：独占查询or定时查询，CPU与设备串行工作



2. 程序中断方式：





中断响应过程：

① 中断隐指令（硬件完成）：关中断 → **保存断点**→ 中断服务程序寻址 （将CPU模式改为内核态）

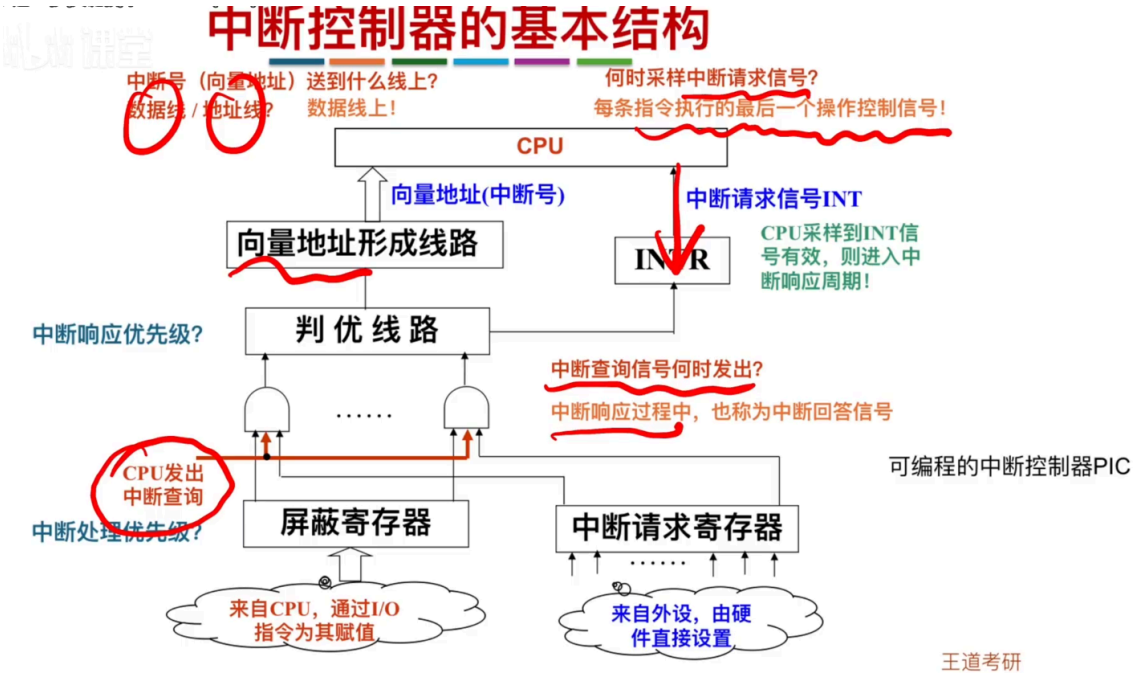
② 中断服务程序（软件完成，即OS）：**保存现场和屏蔽字** → 开中断 → 执行中断服务程序 → 关中断 → 恢复现场和屏蔽字 → 开中断 → 中断返回

其中：

现场：指用户可见的工作寄存器内容，指令可访问，由软件保存到栈/寄存器

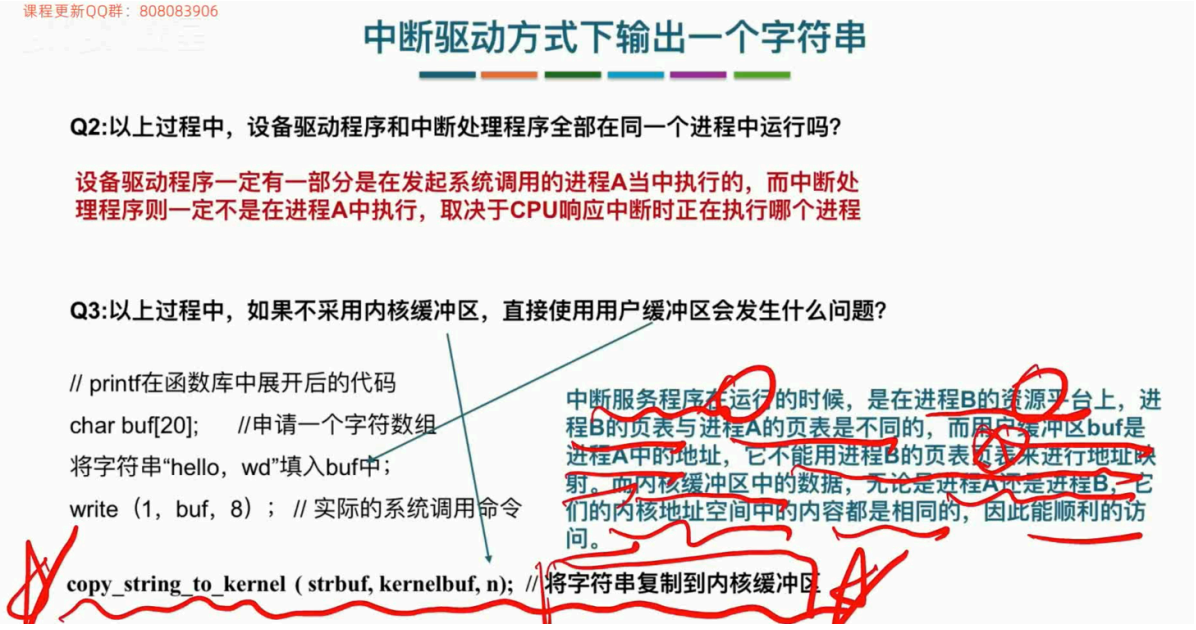
断点：指PC、**PSW**，即返回源程序的地址，指令无法读取，由硬件保存到栈/寄存器

保存屏蔽字即用于中断嵌套，被更高处理优先级的中断给断了之后，要保存本中断的屏蔽字，换成更高的那个

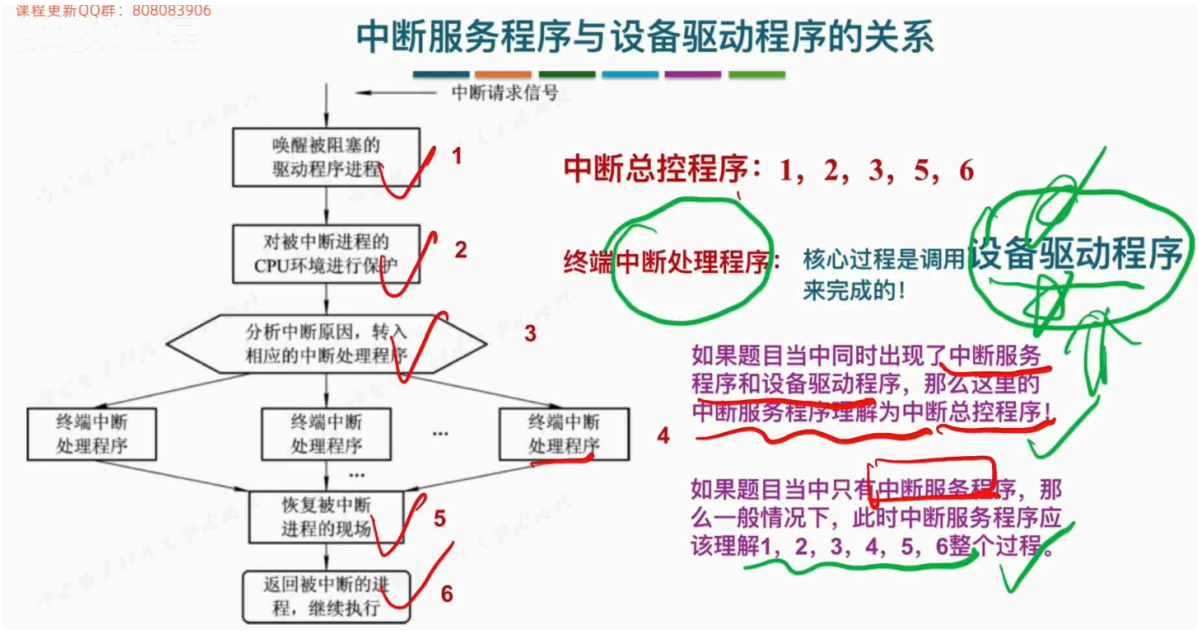


采用中断方式，A进程让打印机输出，然后就阻塞挂起，切换到B进程了，之后打印一个字一个字的中断，都是在B进程的环境下的，所以没有进程切换，实际上中断服务程序就是一段程序，不涉及进程切换。中断服务程序是内核/硬件代码，确实不会发生用户态的进程切换，但是这个中断有可能导致用户态进程切换

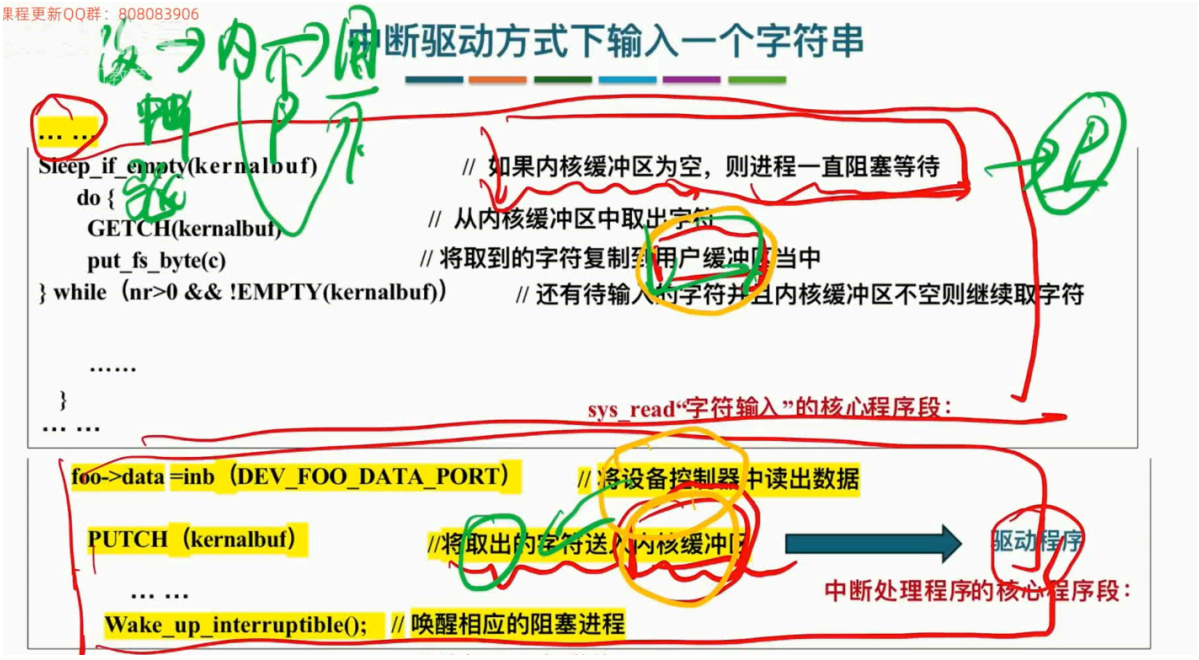
设备驱动程序涉及到的，就是对I/O端口的读写，无论是输出一个字符，还是发送启动命令，解除阻塞等等



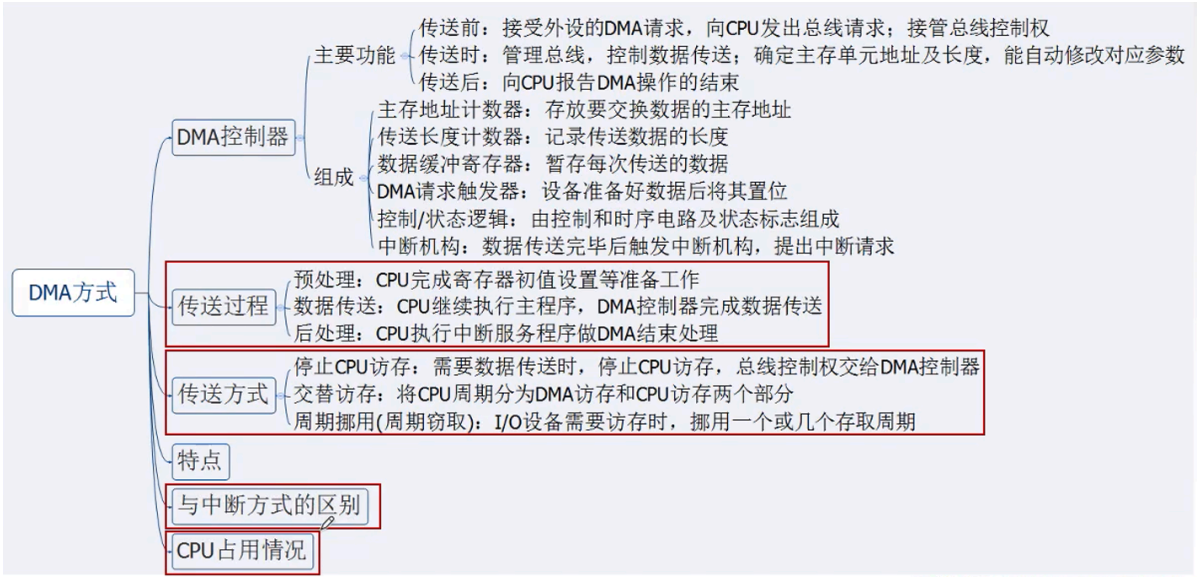
课程更新QQ群：808083906



果程更新QQ群：808083906



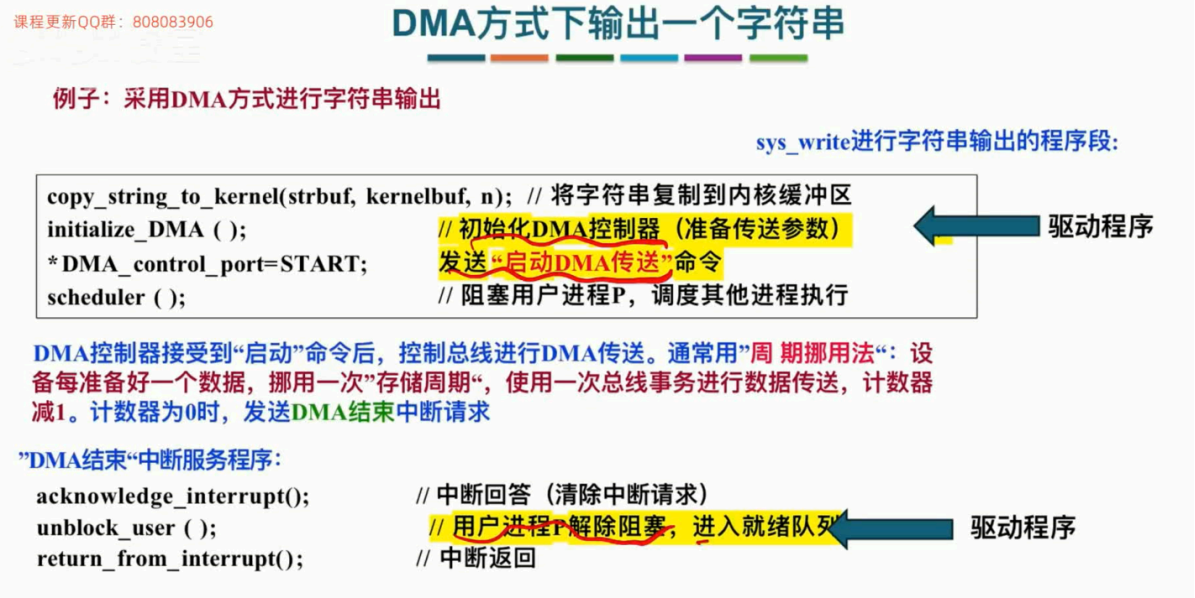
3. DMA方式：



如果是轮询方式，则sys_write系统调用中有while循环，直到所有数据全部输出

如果是中断方式，则sys_write系统调用只输出第一个字符，之后就挂起程序，后续的输出在中断处理程序中完成

如果是DMA方式，则sys_write系统调用只传递参数后启动DMA控制器即可，DMA控制器自己来传输，最后来个结束中断



4. 外部中断指CPU执行指令以外的事件产生的中断，通常来自CPU与内存以外的中断：键盘输入，**时钟中断**，网络数据包到达
5. 若外设准备数据的时间小于中断处理时间，可能导致 外设往缓冲区写入数据的速度 快于 CPU取走的速度，导致缓冲区的数据被覆盖
6. Cache完全由硬件实现，不会涉及中断
7. 浮点数上溢属于内中断，浮点数下溢直接当做机器零，不会中断
8. DMA的优先级比程序中断的优先级更高，DMA在传输过程中不需要保护现场，CPU会在每个存储周期（总线周期）结束后检查是否有DMA请求，**DMA请求的响应在本机器周期结束而非指令结束**，DMA请求是外设发出的
9. 中断优先级：

   ①响应优先级：由中断查询程序/中断判优电路决定，反映多个终端同时请求时哪个先被响应，即中断服务程序**开始执行**的顺序

   ②处理优先级：中断屏蔽标志改变的是处理优先级。在多重中断系统中，决定本中断是否能打断正在执行的中断服务程序，决定的是多个中断服务程序**执行完**的次序
10. 用户程序需要输入输出是，需要调用操作系统提供的接口（请求操作系统服务），此时引起的是访管中断，系统由用户态转为核心态
11. 在开中断情况下，CPU检测到中断就一定会立即响应，因为即使在多重中断中，由于已经检测到了，说明新中断源的处理优先级是更高的，同样会立即响应（优先级更低就不会被检测到）（CPU检测到中断 vs 中断请求发生）